Qualcomm Technologies, Inc.

Qualcomm® Hexagon™ Tensor Processor Overview for SW6100

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

80-74841-45 Rev. AC

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision History

Revision	Date	Description
AC	February 2026	Updated to conform to current document standards. No technical content was changed in this revision.
AB	January 2026	■ Slide 5 *Hexagon Tensor Processor High-level Hardware Architecture*: Updated the Hexagon NPU specifications ■ Slide 8 *Hexagon Tensor Processor – Hexagon DSP*: □ Updated the following specifications: • HTP version • Scalar threads • Processor clock □ Removed the reference chipset SW5100 specifications ■ Slide 9: *Hexagon Tensor Processor – Q6 Core Clock Frequency*: □ Updated the frequency values for SW6100 □ Added a note for clock frequency dependency on SKU □ Removed the reference chipset SW5100 specifications
AA	October 2025	Initial release

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:03 GMT
hyungchul.won@lge.com

Contents

1 Introduction 4

2 Hexagon Tensor Processor 6

3 Hexagon Tensor Processor Hardware Architecture 12

4 Hexagon Tensor Processor Software Architecture 19

5 Hexagon Tensor Processor Use Cases 36

6 Hexagon Tensor Processor Development Environment 39

7 References 67

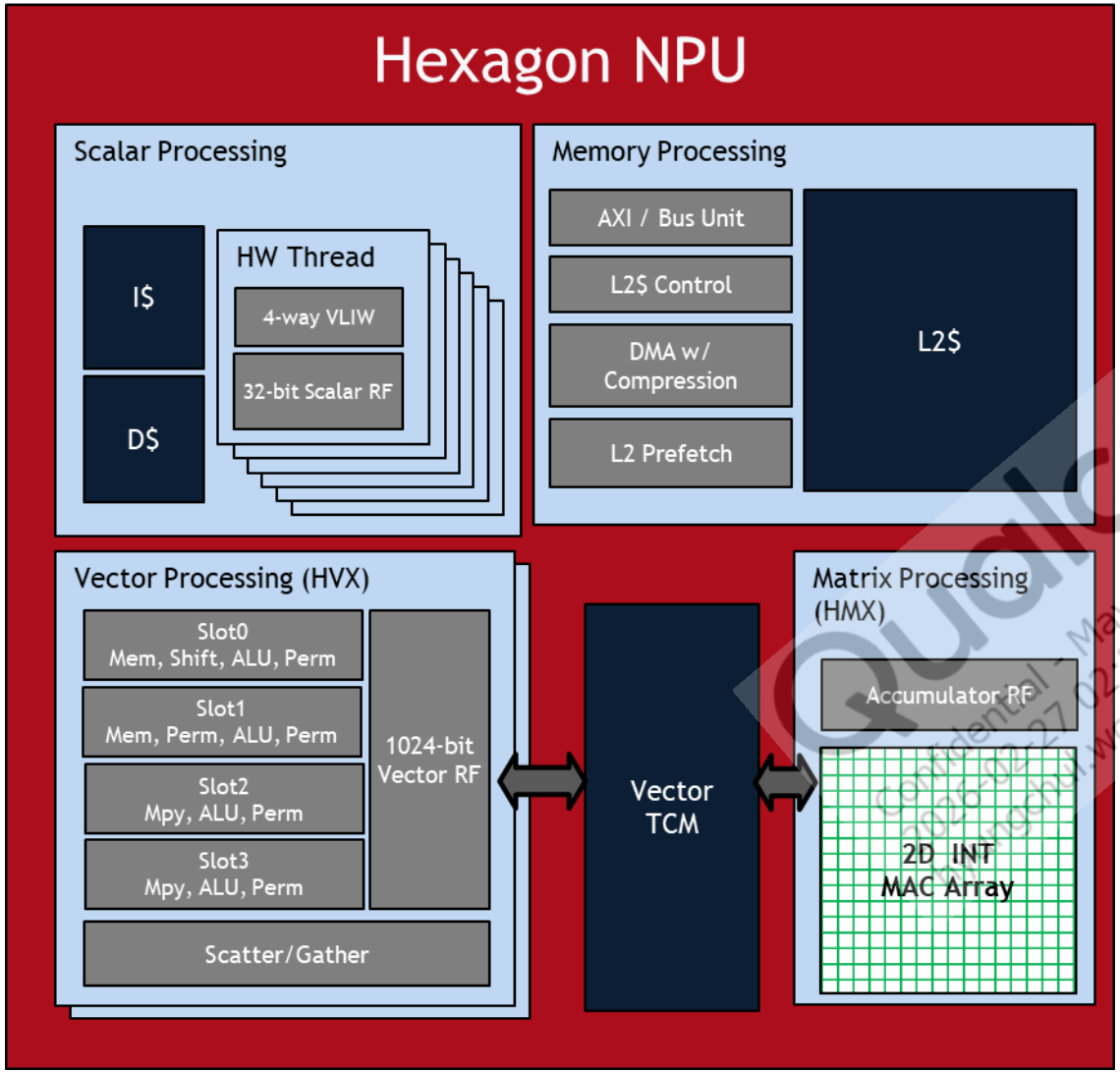
Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Section 1

Introduction

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Hexagon Tensor Processor High-level Hardware Architecture



- Scalar processing NPU6 V81 core with 6 hardware threads in 2 clusters
- 4x Hexagon Vector eXtensions (Qualcomm® Hexagon™ Vector eXtensions (HVX)) vector tightly coupled coprocessors
  - Vector coprocessor optimized for pixel processing
  - Concurrencies where extra threads could be used to run non-ML workloads, support more non-convolutions use case
- 1x Hexagon matrix eXtensions (HMX) matrix tightly coupled coprocessor
  - MAC arithmetic formats, 8-bit and 16-bit integer
- Vector tightly coupled memory (VTCM)
  - 4 MB VTCM
- L2/TCM 1 MB

Qualcomm Confidential – May Contain Trade Secrets 2025-02-27 02:30:08 GMT myungho.won@lge.com

Section 2

Hexagon Tensor Processor

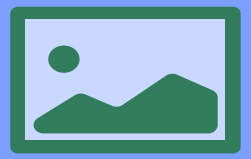
Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Hexagon Tensor Processor – Use Cases

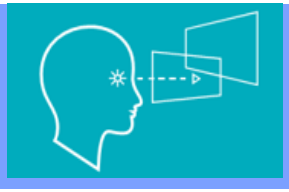
- Hexagon Tensor Processor is dedicated for compute applications
- High performance and low-power engine for machine learning, computer vision, extended reality, and image/video processing

AI and Machine Learning – QNN/SNPE

Image Enhancement – Camera, Still, Video



FastCV and Extended Reality



Video

Development Environment

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.wonalge.com

Hexagon Tensor Processor – Hexagon DSP

	Hexagon Tensor Processor	
	Hexagon V81 (SW6100)	**HVX V81 + Matrix Processing (SW6100)**
Threads/Core	6 scalar threads	▪ + 4 128-byte vector threads ▪ + 1 matrix thread
Processor Clock	▪ 1171.2 MHz Nominal ▪ (1497.6 MHz Turbo_L1)	▪ + 1171.2 MHz Nominal ▪ (+ 1497.6 MHz Turbo_L1)
Arithmetic	Fixed point and floating point	Fixed point and floating point
Caches	▪ L1 Instruction: 32 KB ▪ L1 Data: 16 KB ▪ L2: 1024 KB	Same L2
Architecture	Simultaneous Multi-Threading (SMT)	
DDR Bus Interface	▪ 18 GB per second ▪ Direct interface to DDR	
Instruction/System Enhancements	▪ Multimedia enhanced ▪ Dual-clusters ▪ Coprocessor capability ▪ Automatic Power Collapse (APC)	▪ HVX ▪ Dual-clusters ▪ Scatter/Gather, (VTCM), ▪ DMA, UBWC
	Scalar DSP	

Qualcomm Confidential - May Contain Trade Secrets
2026-2-27 02:30:08 GMT
hyungchul.yun@lge.com

Hexagon Tensor Processor – Q6 Core Clock Frequency

Voltage setting vs. maximum frequency (MHz)	SW6100
TURBO_L1	1497.6
TURBO	1401.6
NOM_L1	1305.6
NOM	1171.2
SVS_L1	960
SVS	787.2
LowSVS	595.2
LowSVS_D1	441.6
LowSVS_D2	249.6

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

NOTES:
- The clock frequency depends on the SKU selected by the user.
- Values are subject to change. See *SW6100 Clock Plan* (80-74841-91).

Hexagon Tensor Processor – Feature Summary

Feature	Description	Documentation source
Compute resource manager	Framework that provides APIs to request and release compute resources (such as HVX, HMX, and VTCM) and enables serialization	`${HEXAGON_SDK_ROOT}\<version>\docs\software\system_performance\resource_management.html`
Multiple protection domains	Ability to separate multiple user processes into memory protection domains (PD)	`${HEXAGON_SDK_ROOT}\<version>\docs\software\system_integration.html`
Unsigned PD	▪ Sandbox for a user compute process that does not require production signature on libraries loaded into it, faster loading time ▪ Recommended PD for most user applications	`${HEXAGON_SDK_ROOT}\<version>\docs\software\system_integration.html`
Signed PD	PD with access to nonsecured drivers that requires loaded libraries to be production-signed. For more details see *Signing: Sub-CA* (80-PD867-113)	`${HEXAGON_SDK_ROOT}\<version>\docs\software\system_integration.html`
Secure PD	▪ Built-in secured PD for trusted applications (that is, biometric use cases) ▪ Supports dynamic loading of algorithm	`${HEXAGON_SDK_ROOT}\<version>\docs\software\system_integration.html` See Software Architecture – Secure PD
Secure PD with TVM	As part of *Qualcomm Camera Security Framework 2.5* (80-40939-151)	
System unsigned PD	A new PD allowed by whitelisted CPU applications that has more access to guest OS services without signature verification	FastRPC, DSP hardware abstraction layer (HAL), SE policy to determine when a whitelisted CPU application requests unsigned.
Debugging tools	Remote debugger, user PD crash log, simulator debug enhancements, PD dump for secured device support	`${HEXAGON_SDK_ROOT}\<version>\docs\tools\debug.html`
Profiling tools – Hexagon trace analyzer	Software trace analysis tool provides increased granularity at function and packet level	`${HEXAGON_SDK_ROOT}\<version>\docs\tools\profile.html`
Sub-CA	Production signing mechanism that replaces CASS or USB token signing	See Development Environment – Sub-CA

Qualcomm Confidential - May Contain Trade Secrets 2025-02-26 02:30:00 GMT hyungchul.wo@lge.com

Hexagon Tensor Processor – Feature Summary (cont.)

Feature	Description	Documentation source
QNN library	▪ Enhanced interface for machine learning, replacing Hexagon NN ▪ Uses new matrix engine and DMA resource to achieve significantly higher throughputs	–
Matrix coprocessor and user DMA	▪ New hardware resources in Hexagon Tensor Processor ▪ Currently only accessible through QNN	–
HVX floating point	Vectorized floating point instructions for half and single precision	`${HEXAGON_SDK_ROOT}\<version>\docs\examples\qhl\README.html`
VTCM	▪ Larger VTCM ▪ Can be partitioned for applications	`${HEXAGON_SDK_ROOT}\<version>\docs\software\system_integration.html#vtcm-partitions`
Asynchronous DSP packet queue	Asynchronous-packet-based, interprocessor communication scheme	`${HEXAGON_SDK_ROOT}\<version>\docs\examples\asyncdspq_example`
Debug config	A config file based on application process : <process_name>.debugconfig	Rpctimeout for hang debugging, callerlevel
DMA buffer heaps	OEM to uses RPCMEM API `rpcmem_alloc()`	–
Version control for skel libraries	▪ The objective of versioning is to enable blacklisting all versions lower than the version specified in oemconfig.so, preventing older libraries from running on targets ▪ All dynamic loading so files should be rebuilt with version tagged	`${HEXAGON_SDK_ROOT}\<version>\examples\calculator\src\version.c`

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 06:30:08 GMT hyungchul.wonghse.com

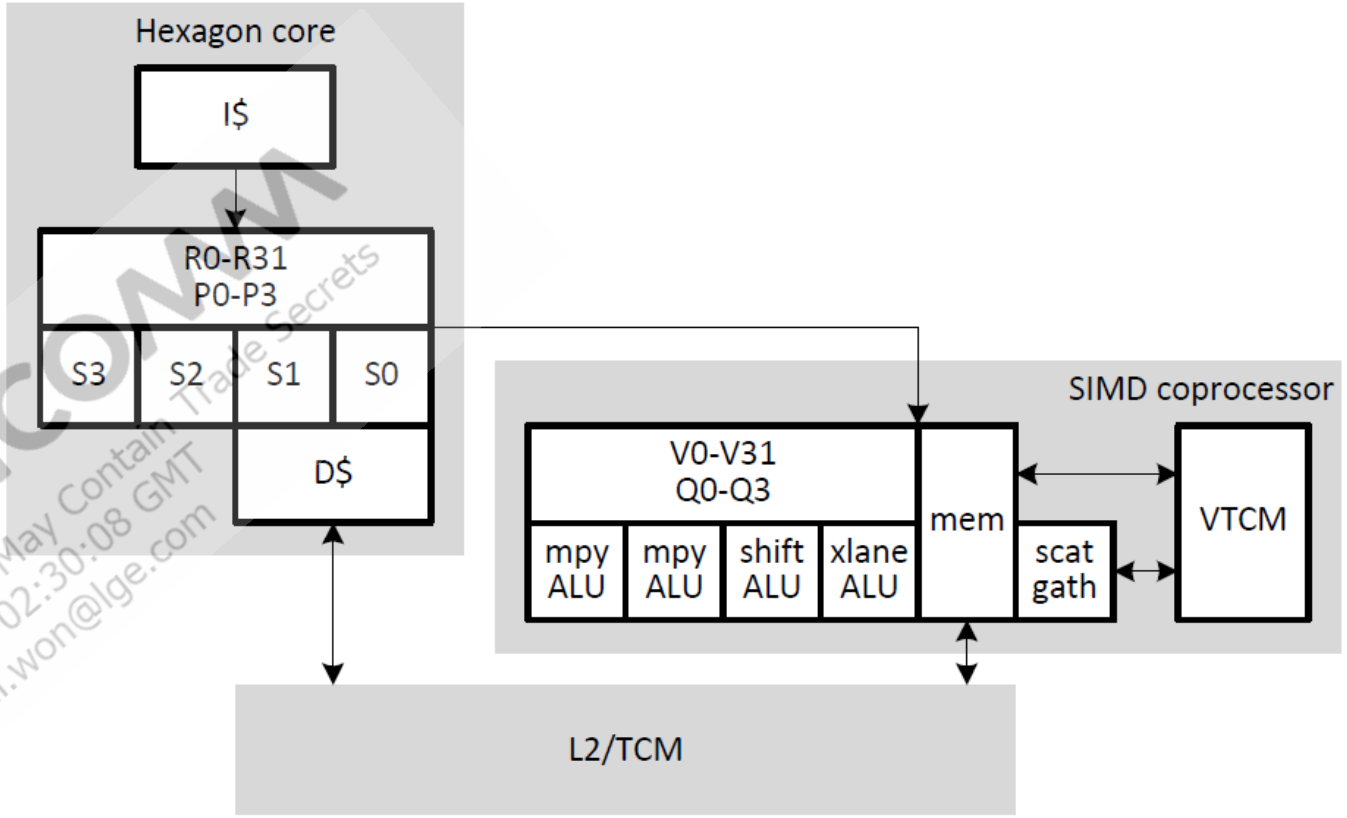
Section 3

Hexagon Tensor Processor Hardware Architecture

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Hexagon Tensor Processor Hardware Architecture – with HVX

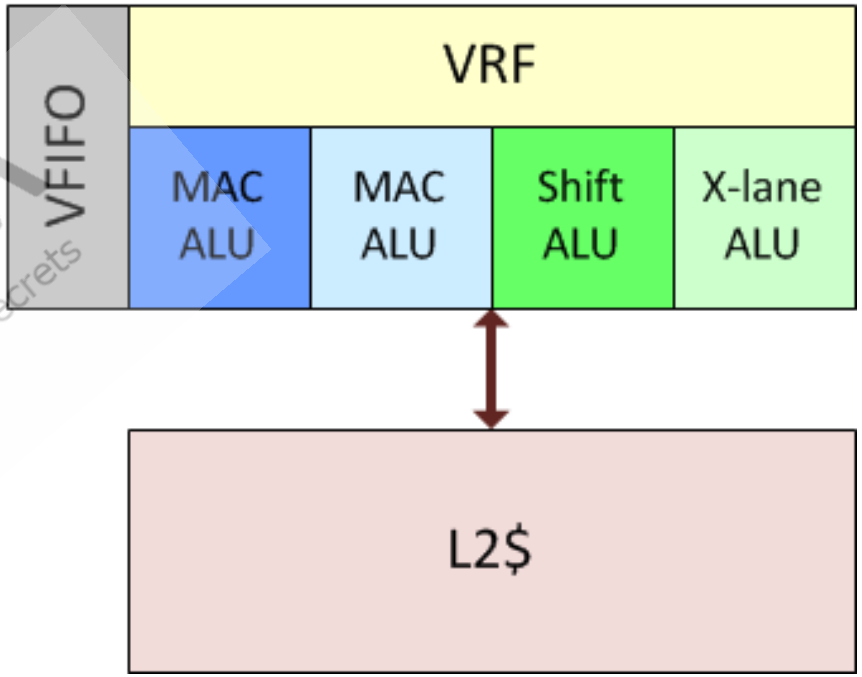
- Interleaved multi-thread Very Long Instruction Word (VLIW) DSP
- Combines best of DSP with general-purpose processor ease-of-programming
- Provides excellent control code and signal processing
- Focuses on throughput and efficiency instead of peak single-thread performance
- HVX is included in Hexagon Tensor Processor supporting vector operations on 1024-bit wide data
- Suitable for high performance imaging, compute, and machine learning applications
- Vectorized floating point instructions for half- and single-precision support added



Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.won@lge.com

Hexagon Tensor Processor Hardware Architecture – with HVX (cont.)

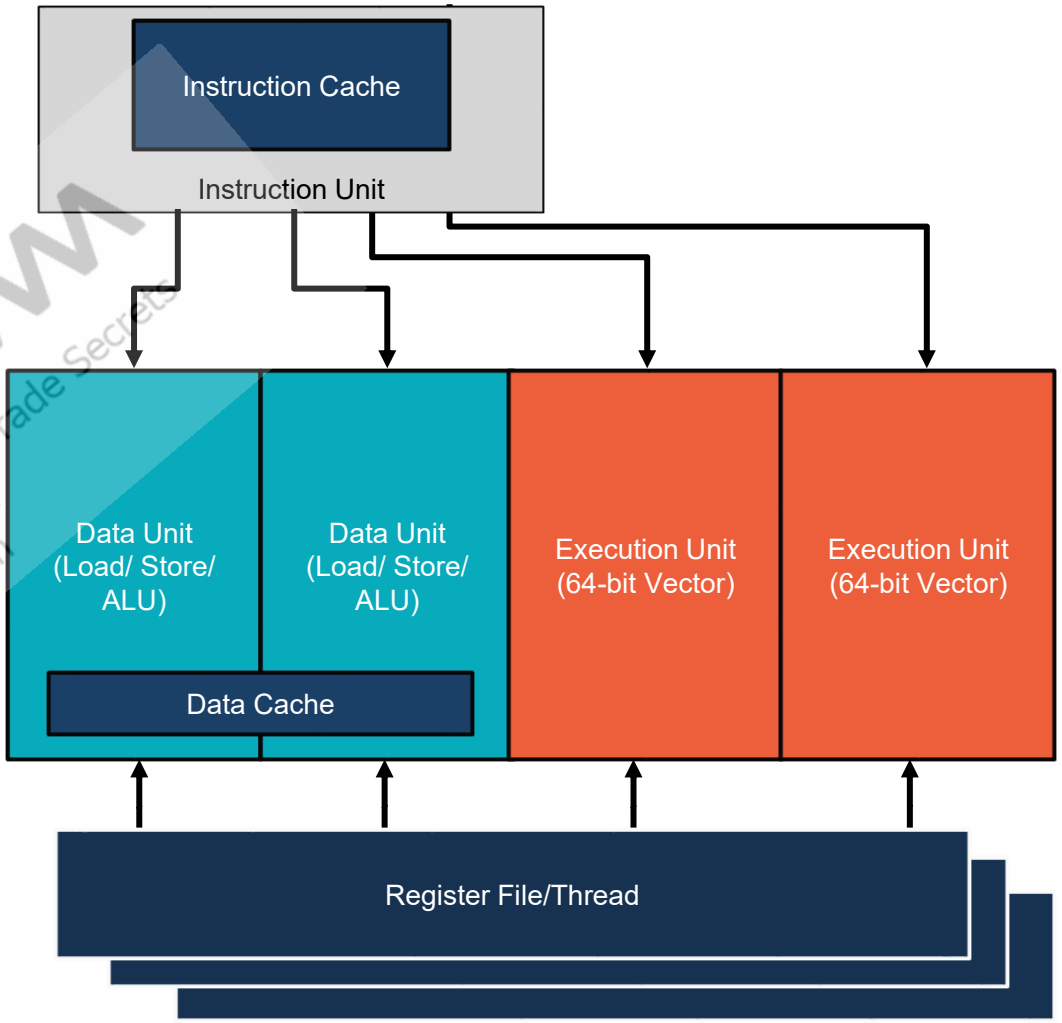
- Four vector execution resources
  - Two multiply-oriented
  - One shift-oriented
  - One permute-oriented
- Simple instructions supported on all resources
  - Add, sub, transfer, logicals
- One load resource + one store resource
  - Access L2 memory directly
  - Kept consistent with L1D$
- VLIW model
  - Compiler/assembly coder chooses which instructions execute in parallel



Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Hardware Architecture – Hexagon Scalar Core

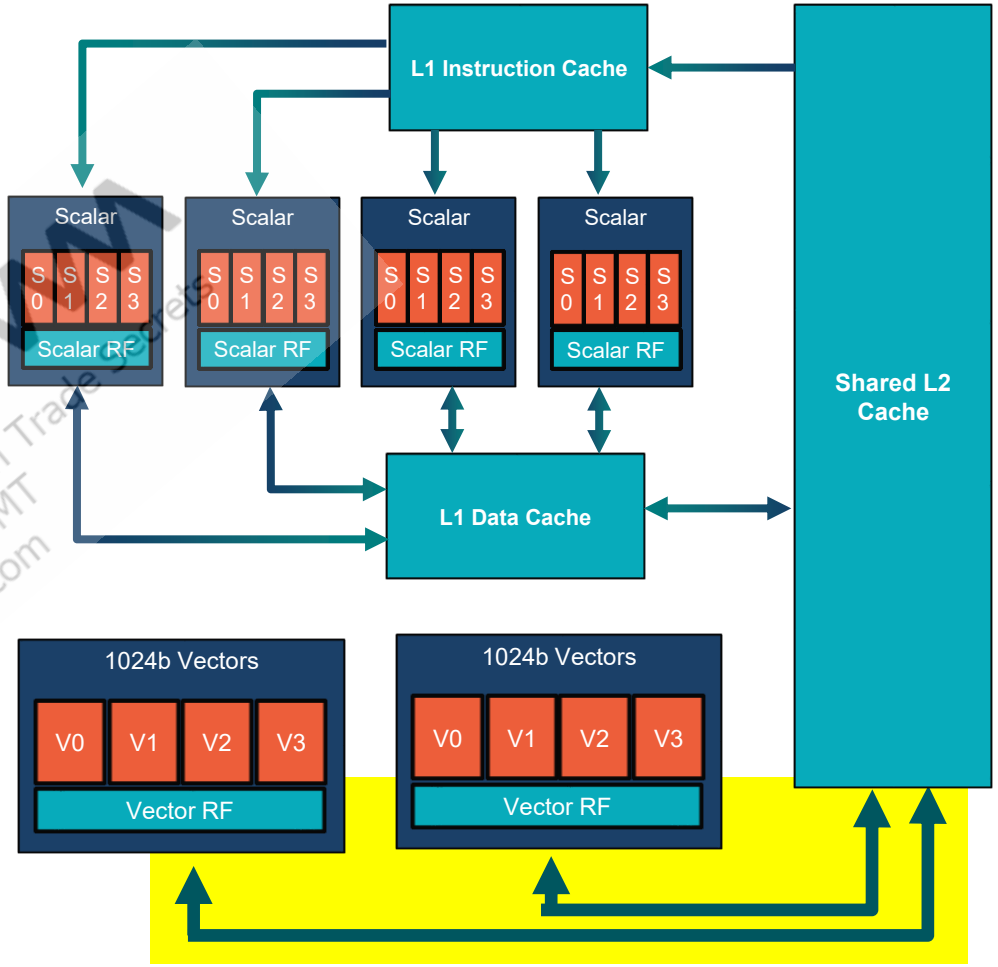
- Orthogonal 4-issue VLIW
- Dual 64-bit load/store units
  - 8/16/32/64-bit load/store, 32-bit scalar arithmetic
- Dual 64-bit vector units
  - 16/32/64-bit vectorized MPY/ALU/SHIFT/FP, permute, bit manipulation, up to 8 MAC/cycle total/2 SP FMA
- Dual branch units
  - Two prioritized compound compare-jumps per cycle
- Unified vector/scalar registers
  - Expands spectrum of code that can be vectorized
- Rich instruction set
- Multiple threads



Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.won@lge.com

Hardware Architecture – HVX Architecture – Memory

- L2 is first-level memory for vector units
  - Large primary memory to hold image data reduces tiling overheads seen on small L1
  - Single cycle load to use
  - Supports full bandwidth
  - Simplifies programming
- L1 and L2 are hardware coherent
- Streaming prefetch from DDR to L2
- Vector units support a variety of load/store instructions
  - Unaligned
  - Per-byte conditional



Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Instruction Packets

- Instructions can be grouped into VLIW packets for parallel execution, with each packet containing from one to four instructions
- Packets of varying length can be freely mixed in a program
- Vector instructions operate on single instruction multiple data vectors
  - For example, two instructions grouped in a packet:

```
{
   R0 = R1; R2 = R3
}
```

  - Four instructions grouped in a packet (scalar)

```
{ R8 = memh(R3++#2);
  R12 = memw(R1++#4);
  R0 = mpy(R10,R6):<<1:sat;
  R7 = add(R9,#2);
}
```

  - Four instructions grouped in a packet (HVX)

```
{ V0 = VMEM(iptr0++#1)
  V12.h = VADD(sVsum0.h,sVsumv0O.h)
  V6.h += VMPYI(sVsumv0O.h,const2.b)
  V11 = VALIGN(sVsumv2E,sVsumv1E,#2)
}
```

Vector instructions

```
int i;
Word64 *vIn1 = (Word64*)In1;
Word64 *vIn2 = (Word64*)In2;
Word64 *vOut = (Word64*__restrict)Out;
for (i = 0; i < samples/4; i++) {
  vOut[i] = Q6_P_vaddh_PP(vIn1[i], vIn2[i]);
}
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@live.com

Registers

- HVX registers (1024 bits)

```
V0 :
  ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff
  ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff
  ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff
  ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff,ffffffff
V1 :
  1010101,1010101,1010101,1010101,1010101,1010101,1010101,1010101
  1010101,1010101,1010101,1010101,1010101,1010101,1010101,1010101
  1010101,1010101,1010101,1010101,1010101,1010101,1010101,1010101
  1010101,1010101,1010101,1010101,1010101,1010101,1010101,1010101
V2 :
  2,2,2,2,2,2,2,2
  2,2,2,2,2,2,2,2
  2,2,2,2,2,2,2,2
  2,2,2,2,2,2,2,2
V3 :
  3,3,3,3,3,3,3,3
  3,3,3,3,3,3,3,3
  3,3,3,3,3,3,3,3
  3,3,3,3,3,3,3,3
V4 :
  4,4,4,4,4,4,4,4
  4,4,4,4,4,4,4,4
  4,4,4,4,4,4,4,4
  4,4,4,4,4,4,4,4
```

- Scalar vector instructions (four data elements at a time)

```
int i;
Word64 *vIn1 = (Word64*)In1;
Word64 *vIn2 = (Word64*)In2;
Word64 *vOut = (Word64*__restrict)Out;
for (i = 0; i < samples/4; i++) {
  vOut[i] = Q6_P_vaddh_PP(vIn1[i], vIn2[i]);
}
```

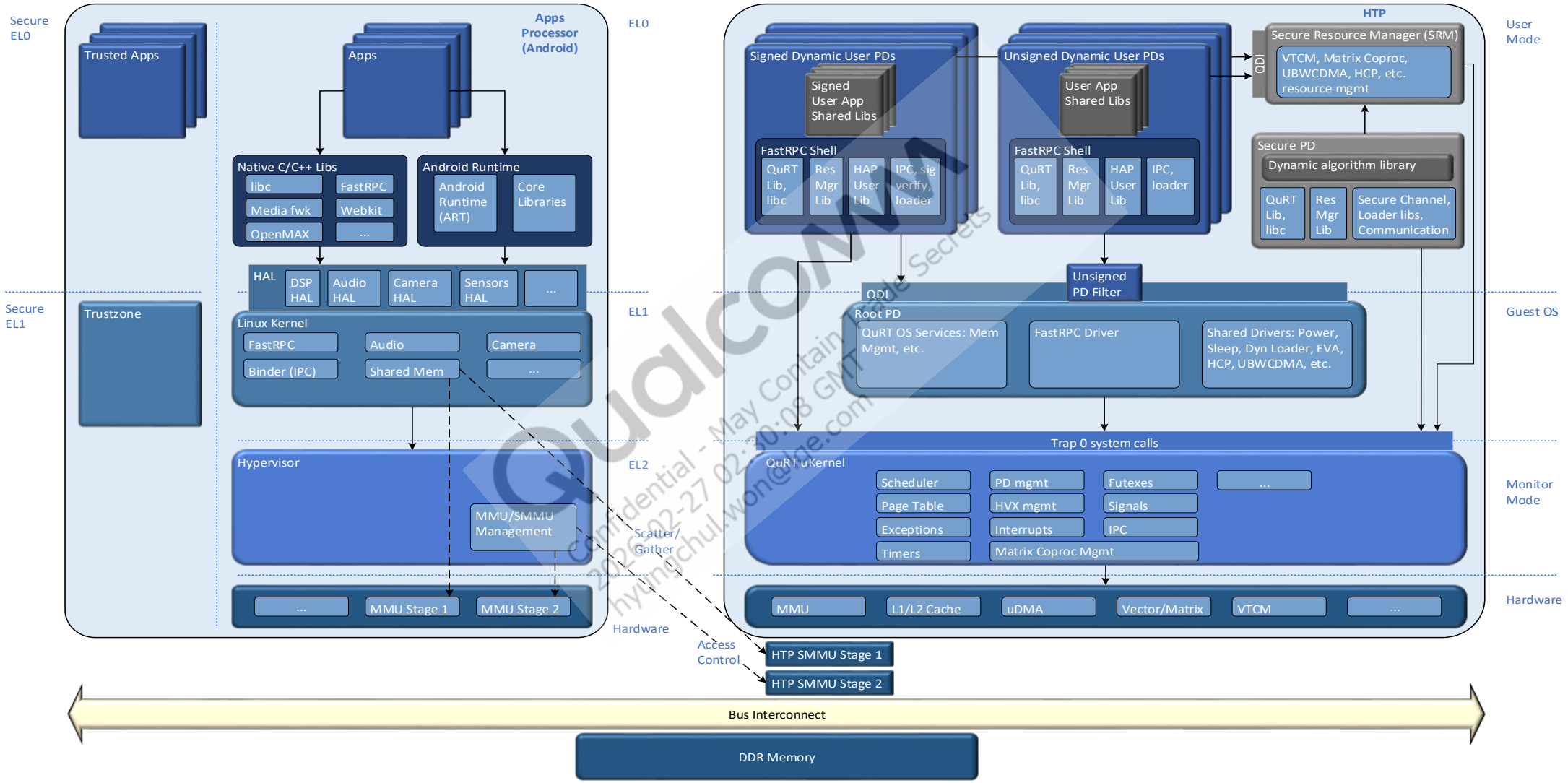
Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.won@lge.com

Section 4

Hexagon Tensor Processor Software Architecture

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Hexagon Tensor Processor Android Software Architecture



Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 02:50:08 GMT hyungchul.won@qce.com

Hexagon Tensor Processor Software Architecture

The following are the building blocks in the Hexagon Tensor Processor software architecture:

- Application processor
  - User applications (trusted and untrusted applications)
  - FastRPC user space and kernel drivers
  - DSP HAL
  - Shared memory for IPC (Inter Process Communication)
- Hexagon Tensor Processor
  - Dynamic signed and unsigned PDs
  - Secure PD
  - Root PD
    - QuRT™ Software OS, Memory Management, FastRPC, power, sleep drivers, and so forth
  - Secure Resource Manager
  - QuRT µKernel

For details on the software architecture, see Hexagon SDK documentation at

`${HEXAGON_SDK_ROOT}\<version>\/docs/software`

Qualcomm
Confidential – May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@qce.com

Software Architecture – QuRT RTOS

- QuRT OS is a Qualcomm real-time operating system that runs on DSPs and supports multi-threading, thread communication and synchronization, interrupt handling, and memory management
- For details, see Hexagon SDK
  `@${HEXAGON_SDK_ROOT}\<version>\docs/software/os/os_support_dsp.html`

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Software Architecture – Power Management (CDSPPM), DCVS, and HAP Utility

- Hexagon Tensor Processor and bus clocks are managed within Hexagon Tensor Processor by a combination of voting (CDSPPM) and background dynamic clock and voltage scaling (DCVS) task
  - Voting establishes initial clocks
    - Floor clocks are guaranteed throughout voting use case life cycle
  - CDSPPM aggregates client votes for Hexagon core clock and bus clock to DDR
    - Sets default clock to nominal (NOM) frequency
  - DCVS monitors processor loading
    - Might raise or lower clocks to meet real-time needs with lowest-possible power
    - Force DSP sleep by overriding sleep latency voting if there is no activity for 500 ms
- Hexagon Access Program (HAP) utility APIs provide access for setting clock and bus votes together with other DCVS parameters

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Software Architecture – Power Management – DCVS v3 API (HAP Utility API)

- Method to set power values from the DSP

```
HAP_power_set (void* context, HAP_power_request_t* request)
```

- Used to enable/disable DCVS

```
HAP_power_set_DCVS_v3  (request type)
```

- For HAP API details, see Hexagon SDK documentation at:

```
${HEXAGON_SDK_ROOT}\<version>\docs\doxygen\HAP_power\index.html
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

DCVS Aggregation Result

AGGREGATED_TARGET_CORNER = ADSP_DCVS_VCORNER_SVS

|-> Aggregated core clock vote to DSPPM: 577000KHz

|-> Aggregated bus clock vote to DSPPM: 465501KHz

AGGREGATED_MIN_CORNER = ADSP_DCVS_VCORNER_SVS2

AGGREGATED_MAX_CORNER = ADSP_DCVS_VCORNER_NOM

Aggregated DCVS state request is Enabled

Aggregated DCVS Policy = ADSP_DCVS_POLICY_POWER_SAVER

Aggregated_client_class = MMPM_COMPUTE_CLIENT_CLASS,

Aggregated Sleep Latency vote to DSPPM: 100

clientId = 5
TID = 0x1F7C
PID = 0x84
PROCESS NAME = CamXBITMLInit
HLOS PID = 0x548
ASID = 0x3
THREAD NAME = fr/1352/31811
DCVS PARTICIPATION Disabled
NO sleep latency vote
NO corner votes

No voting

clientId = 7
TID = 0x1F7C
PID = 0x84
PROCESS NAME = CamXBITMLInit
HLOS PID = 0x548
ASID = 0x3
THREAD NAME = fr/1352/31811
DCVS PARTICIPATION Enabled
DCVS Policy = ADSP_DCVS_POWER_SAVER_MODE
SLEEP LATENCY = 100
TARGET_CORNER = ADSP_DCVS_VCORNER_SVS
MIN_CORNER = ADSP_DCVS_VCORNER_SVS2
MAX_CORNER = ADSP_DCVS_VCORNER_NOM

dominated voting from client #7

clientId = 6
TID = 0x380
PID = 0x84
PROCESS NAME = CamXBITMLInit
HLOS PID = 0x548
ASID = 0x3
THREAD NAME = fr/1352/3260

DCVS PARTICIPATION Enabled
DCVS Policy = ADSP_DCVS_POWER_SAVER_MODE
SLEEP LATENCY = 65535
TARGET_CORNER = 0
MIN_CORNER = 0
MAX_CORNER = 0

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:38 GMT hyungchul.kim@qualcomm.com

Software Architecture – User Application (Hexagon SO)

- Hexagon and HVX functions are built with toolchain in Hexagon SDK and placed on device file system
- Loaded dynamically when invoked by user application on application processor
- Invokes HAP APIs with DCVS v3 parameters for required clock corner, DDR bandwidth, and DCVS enable/disable
- Supports C and C++ applications

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Software Architecture – FastRPC

- A remote procedure call (RPC) allows a computer program calling a procedure to execute in another remote processor, while hiding the details of the remote interaction

  FastRPC is the RPC mechanism used to enable remote function calls between the CPU and DSP

- For details, see Hexagon SDK documentation at:

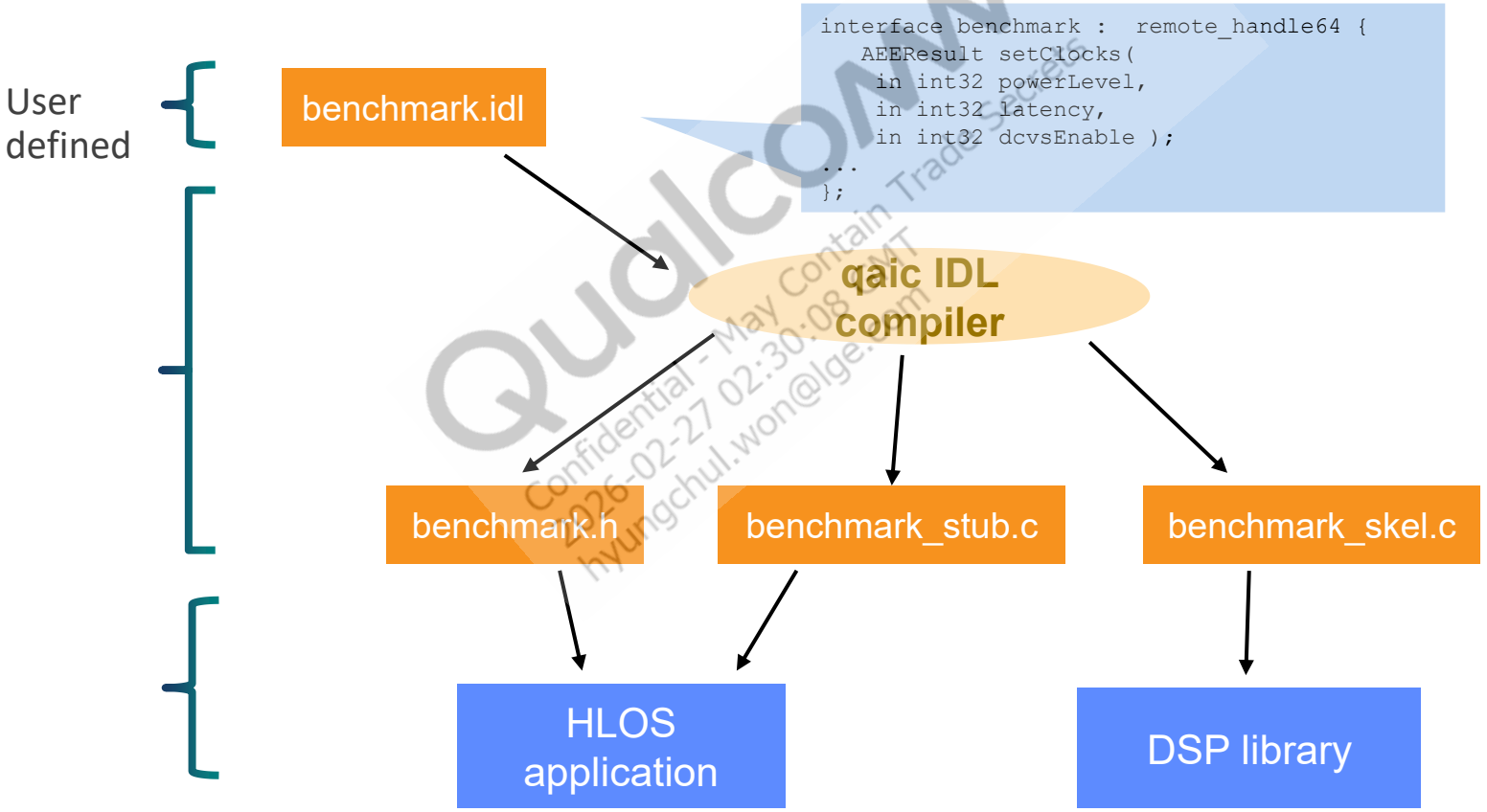
  `${HEXAGON_SDK_ROOT}\<version>\docs/software/ipc/rpc.html`

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Software Architecture – FastRPC – IDL (Interface Description Language)

- IDL file defines interfaces that are remoted to CPU and implemented on DSP
- Qaic compiles IDL files into headers, stubs, and skels
- For more information on IDL compiler, see Hexagon SDK documentation:

  `${HEXAGON_SDK_ROOT}\<version>\docs/reference/idl.html`



Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.won@lge.com

Software Architecture – FastRPC – remote_session_control

- Sets remote session parameters, such as thread stack size or unsigned PD mode. Enables killing the remote process, closes sessions on the DSP, generates a PD dump, or triggers remote process exceptions.
- Usage example to crash cDSP
  - **Note:** Before invoking `remote_handle64_control`, handle `h` must be acquired first using `remote_handle64_open`.

```
#include "remote.h"

remote_rpc_process_exception data;
data.domain = CDSP_DOMAIN_ID;
int nErr;
if ((nErr = remote_session_control(FASTRPC_REMOTE_PROCESS_EXCEPTION, (void*)&data,
sizeof(data)))) {
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:06 GMT hyungchul.won@lge.com

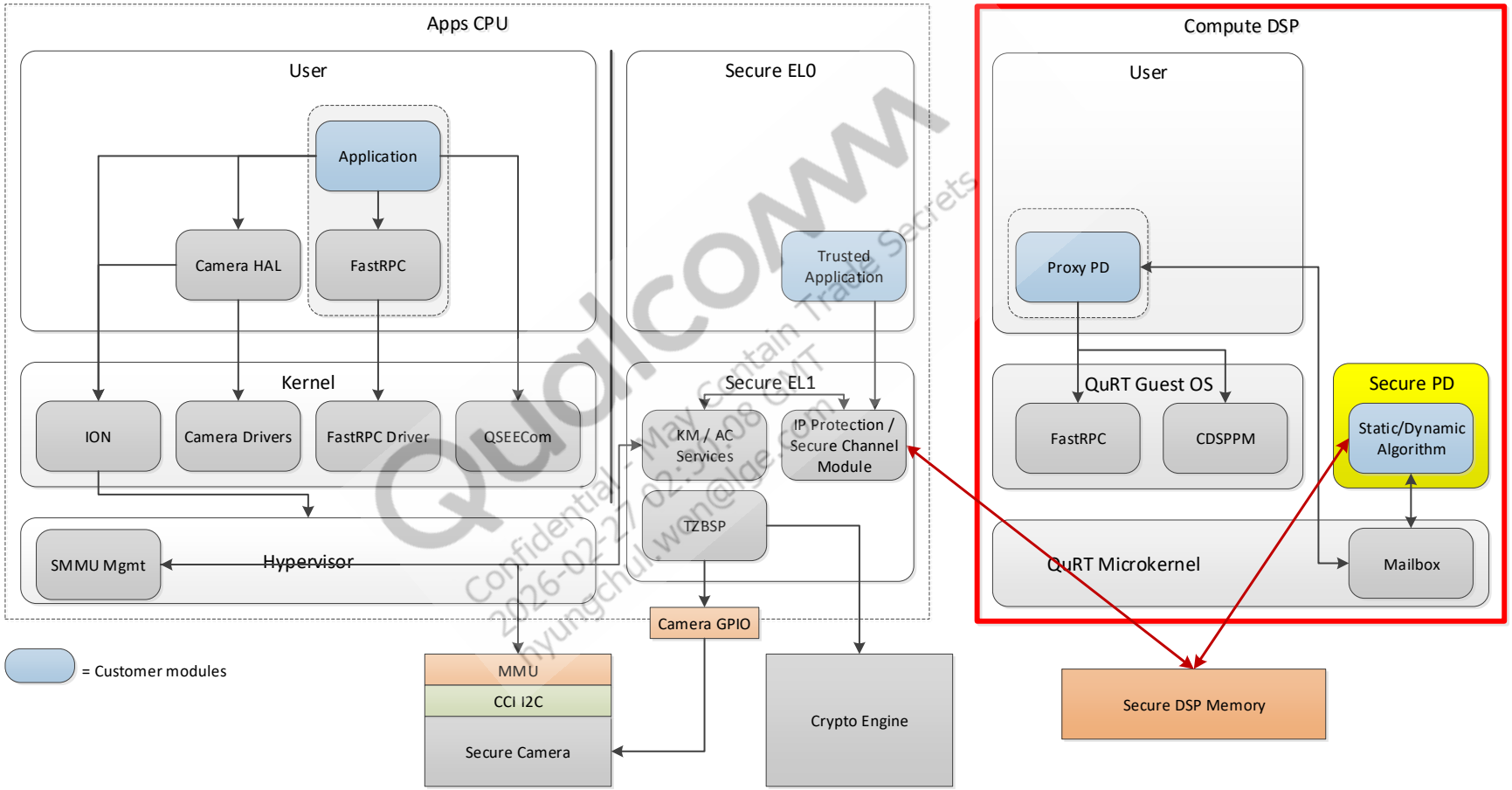
Software Architecture – FastRPC Voting

- With the evolving compute use cases, the FastRPC (HAP) client must vote the clock and DDR bandwidth
  - Every HAP power client must use a unique context by using `HAP_utils_create_context/HAP_utils_destroy_context`
  - FastRPC now casts a temporary clock boosting vote at the beginning of each new session
    - Startup boost vote is permanently removed, as soon as any client has completed explicit voting
    - Startup boost vote is not restored after the client removes their vote
  - FastRPC also votes for temporarily boosted clocks during object loading or other significant operations

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Software Architecture – Secure PD (with TZ)

- Static – Algorithm shared object compiled as part of DSP image in SecurePD
- Dynamic – Algorithm shared objects can be loaded dynamically in SecurePD

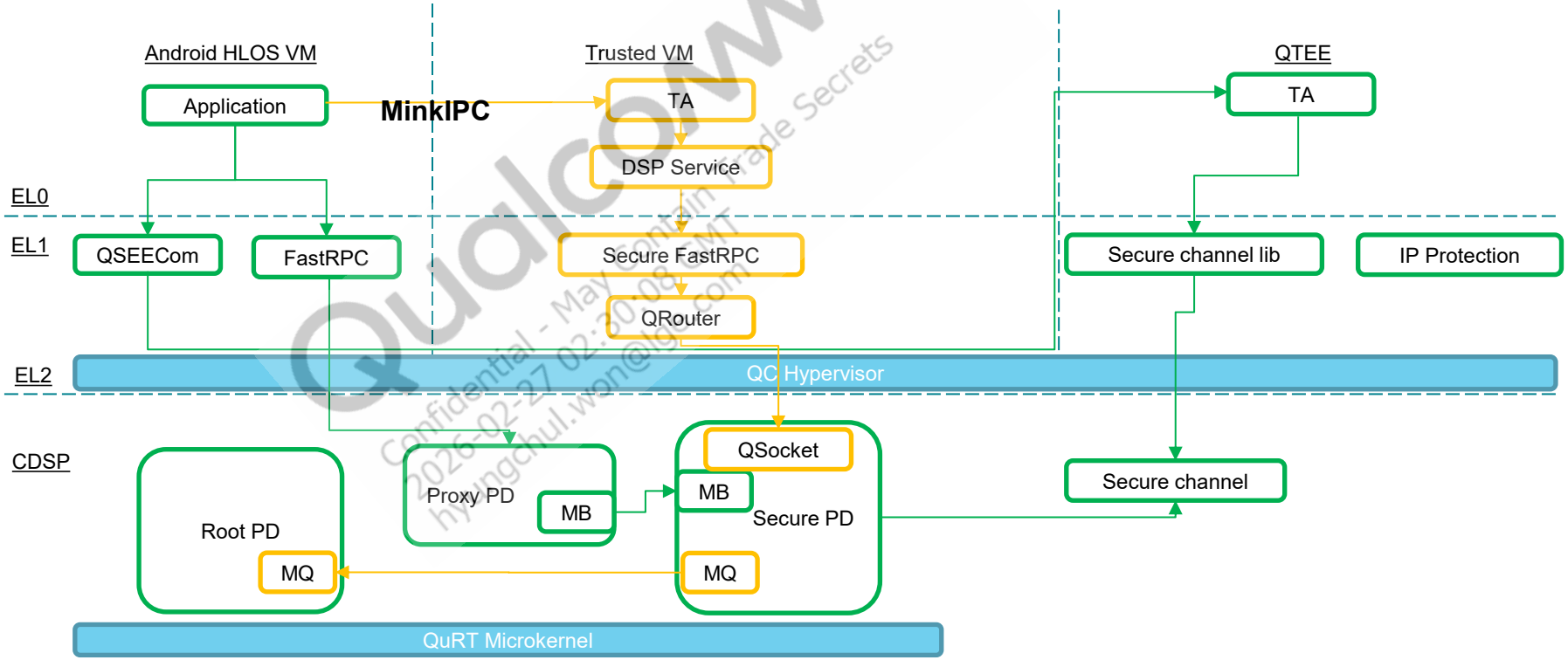


Note: See *Dynamic Loading in Secure PD* (80-PR150-1) and *Hexagon Secure cDSP Software Overview for SM8150* (80-PK723-1).

Software Architecture – Secure PD (with TVM)

The secure PD call flow includes:

- PVM app, TVM TA, TVM DSP_service TVM fastrpc driver, secure PD on CDSP
- Embedded prebuilt DSP_service running on TVM listening for requests from Tas
- `frpc-trusted-adsprpc.ko` kernel driver and `libcdsprpc.so` from LE release
- For more details, see *Qualcomm Trusted VM Platform* (80-68347-1) and *Qualcomm Camera Security Framework 2.5* (80-40939-151)



TVM solution in Orange

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.won@lge.com

Secure PD (with TVM) - Example and Testing

- `adb shell "loadalgo -t /data/local/tmp/Loadalgo_TA -e /data/local/tmp/example_image_runner.so"`
- LA Vendor
  - `dspservices_ship\loadalgo_example`
    - `loadalgo.c loadalgo_qtvm.cpp`
- Linux Embedded
  - Example code
    - `apps_proc\src\vendor\qcom\proprietary\dspservices_ship\loadalgo_ta_tvm`
      - `CLoadalgo_TA.cpp`
    - `apps_proc\build-qti-distro-base-user\tmp-glibc\deploy\images\trustedvm-v2\trustedvm\Loadalgo_TA`
- CDSP example algo running on Secure PD
  - `cdsp_proc\platform\securepd\example\gaussian\src`
  - `cdsp_proc\build\ms\dynamic_modules\product.cdsp.prod\example_image_runner.so`

Qualcomm Confidential - May Contain Trade Secrets 2025-02-17 02:30:00 GMT hyungchul.won@lge.com

Secure PD (with TVM) – Android Logging

```c
*/
int run_gaussian_algo_qtvm(char* TA_path, char* module_path, unsigned dynamic_heap_size, unsigned hlos_physpool_size) {
   Loadalgo_TA *TA_session_handle = nullptr;
   int32_t nErr = 0;
   minkIPC_connection mink_connect;
   file_info runtime_farf_file_info = {};

   printf("%s: running through QTVM\n", __func__);
   memset(&mink_connect, 0, sizeof(mink_connect));
   if (module_path == nullptr) {
      nErr = AEE_EBADPARM;
      ALOGE("Error 0x%x: %s: incorrect module name\n", nErr, __func__);
      goto ↓bail;
   }
   nErr = setup_minkIPC_connection(&mink_connect);
   if (nErr) {
      goto ↓bail;
   }

   TA_session_handle = create_TA_IPC_session(&mink_connect, TA_path, CLoadAlgoTA_UID);
   if (TA_session_handle == nullptr) {
      nErr = AEE_EMEMPTR;
      goto ↓bail;
   }

   nErr = enable_runtime_farf(TA_session_handle, &runtime_farf_file_info);
   if (nErr) {
      goto ↓bail;
   }

   nErr = start_algo(TA_session_handle, module_path, dynamic_heap_size, hlos_physpool_size);
   if (nErr) {
      goto ↓bail;
   }
```

```c
*/
static int32_t start_algo(Loadalgo_TA *TA_session_handle, char *module_path, unsigned dynamic_heap_size, unsigned hlos_physpool_size) {
   int32_t nErr = AEE_SUCCESS, str_len = 0;
   file_info module_info = {};
   char secure_module_interface[SECURE_MODULE_INTERFACE_LENGTH + 1] = {};

   memset(&module_info, 0, sizeof(module_info));
   module_info.name_length = strlen((const char*)module_path);
   if (module_info.name_length + 1 > NAME_SIZE) {
      nErr = AEE_EBUFFERTOOSMALL;
      goto ↓bail;
   }
   strlcpy(module_info.name, (const char*)module_path, sizeof(module_info.name));
   nErr = prepare_module(&module_info);
   if (nErr) {
      goto ↓bail;
   }

   str_len = snprintf(secure_module_interface, SECURE_MODULE_INTERFACE_LENGTH + 1, "%s", SECURE_MODULE_INTERFACE);

   nErr = TA_session_handle->run_algo(*module_info.proxy_file, (uint32_t)module_info.file_buffer->size, (const uint8_t*)secure_module_interface, str_len,
                                      dynamic_heap_size, hlos_physpool_size);
   printf("%s: run_algo returned 0x%x(%d)\n", __func__, nErr, nErr);
bail:
   if (nErr)
      printf("Error 0x%x(%d): %s: failed for module %s\n", nErr, nErr, __func__, module_path);

   secure_mem_free(module_info.file_buffer);
   FREEIF(module_info.file_buffer);
   DELETEIF(module_info.proxy_file);
   return nErr;
} « end start_algo »
```

```
pineapple:/data/local/tmp # loadalgo -t Loadalgo_TA -e example_image_runner.so
main: loadalgo_TA abs path Loadalgo_TA
main: running gaussian algo elf and exiting after run
run_gaussian_algo_qtvm: running through QTVM
setup_minkIPC_connection: Connection with Mink QRTR LE service 5008 established
create_TA_IPC_session: opened session to process loader service (uid 324)
create_TA_IPC_session: file Loadalgo_TA (size 2097152 bytes) loaded to fd object using heap 3
create_TA_IPC_session: service Loadalgo_TA loaded on QTVM
create_TA_IPC_session: TA service (uid 401) connection established with session handle 0xb400007d7b3d8520
enable_runtime_farf: setting runtime mask to secure PD
prepare_module: created ProxyBase object for module example_image_runner.so using heap 2
start_algo: run_algo returned 0x0(0)
run_gaussian_algo_qtvm: TA session handle 0xb400007d7b3d8520 destroyed
run_gaussian_algo_qtvm: Test returned 0x0
main: run_gaussian_algo returned 0x0 (0)
retry_sequence_after_ssr: SecurePD test Success
main: HLOS securePD test app exiting with nErr 0x0
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT kyungchul.yoon@lge.com

Secure PD (with TVM) – QTVM Logging

```
int32_t CLoadalgo_TA::run_algo(const ProxyBase &soFile_ref, uint32_t soFileSize_val, const uint8_t* moduleName_ptr, size_t moduleName_len, uint32_t heapSize_val, uint32_t poolSize_val) {
    int32_t nErr = SUCCESS, ret = SUCCESS;
    ProxyBase *session_proxy = nullptr;
    ITEnv *my_opener = nullptr;
    SecureDspSession* dsp_service_session_handle = nullptr;
    SecureDspModule* dsp_module_session_handle = nullptr;
    module_op operation = GAUSSIAN;

    if (!heapSize_val || !poolSize_val) {
        nErr = EBADSIZE;
        LOG_QTVM("Error %d: %s: heap size %u or pool size %u are invalid\n", nErr, __func__, heapSize_val, poolSize_val);
        goto ↓bail;
    }
    my_opener = new ITEnv(gTVMEnv);
    if (my_opener == nullptr) {
        nErr = EMEMPTR;
        LOG_QTVM("Error 0x%x: %s: failed to open QTVM gTVMEnv\n", nErr, __func__);
        goto ↓bail;
    }

    dsp_service_session_handle = create_dsp_service_session(&session_proxy, my_opener, &this->farf_buffer);
    if (dsp_service_session_handle == nullptr) {
        nErr = EMEMPTR;
        goto ↓bail;
    }

    nErr = alloc_memory_in_dsp_session(dsp_service_session_handle, heapSize_val, poolSize_val);
    if (nErr) {
        goto ↓bail;
    }

    dsp_module_session_handle = load_module_in_dsp_session(dsp_service_session_handle, &soFile_ref, soFileSize_val, moduleName_ptr, moduleName_len);
    if (dsp_module_session_handle == nullptr) {
        nErr = EMEMPTR;
        goto ↓bail;
    }

    nErr = run_cmd_on_DSP_service(dsp_module_session_handle, my_opener, operation);
    if (nErr)
        goto ↓bail;

bail:
    ret = close_module_in_dsp_session(dsp_service_session_handle, dsp_module_session_handle);
    if (!nErr)
        nErr = ret;
    destroy_dsp_service_session(dsp_service_session_handle, session_proxy);
    DELETEIF(dsp_module_session_handle);
    DELETEIF(my_opener);

    if (nErr) {
        LOG_QTVM("Error 0x%x(%d): %s: failed load/run operation %u for module interface %s\n", nErr, nErr, __func__, operation, moduleName_ptr);
    }
    return nErr;
} « end run_algo »
```

```
trustedvm-v2 TVMRoot[704]: DSP_service: 0x0: main: env DSP_LIBRARY_PATH set to /tmp
trustedvm-v2 TVMRoot[704]: DSP_service: 0x0: setup_securepd_session_sem: done
trustedvm-v2 TVMRoot[704]: DSP_service: 0x0: setup_mem_service_pool: setup memory pool service for uid 325
trustedvm-v2 TVMRoot[704]: DSP_service: 0x0: main: service initialized. Waiting for requests
trustedvm-v2 TVMRoot[704]: DSP_service: 0x7f98001220: CSecureDspSession: Creating DSP service session 0x7f98001220
trustedvm-v2 TVMRoot[704]: DSP_service: 0x0: get_credentials: client name Loadalgo_TA
trustedvm-v2 TVMRoot[704]: DSP_service: 0x0: get_credentials: distinguished ID 202
trustedvm-v2 TVMRoot[704]: DSP_service: 0x0: tProcessOpen: DSP service session created. DSP service session handle 0x7f98001220, service uid 400
trustedvm-v2 TVMRoot[704]: DSP_service: 0x7f98001220: enableRuntimeFARF: Created file /tmp/DSP_service.farf (511 bytes) with content 0x1f001f00000000
trustedvm-v2 TVMRoot[704]: ADSPRPC: 2 3 I DSP_service: rpcmem_qtvm.cpp:107: setup_mem_service_pool: setup memory pool service for uid 325
trustedvm-v2 TVMRoot[704]: ADSPRPC: 2 3 I DSP_service: rpcmem_qtvm.cpp:149: create_memory_pool: user pool created size 307200
trustedvm-v2 TVMRoot[704]: ADSPRPC: 2 3 I DSP_service: fastrpc_config.c:319: Reading configuration file: DSP_service.debugconfig
trustedvm-v2 TVMRoot[704]: ADSPRPC: 2 3 I DSP_service: fastrpc_config.c:350: fastrpc_config_init: Couldn't find file DSP_service.debugconfig, errno (No such file or directory) at /tmp,
trustedvm-v2 TVMRoot[704]: ADSPRPC: 2 3 I DSP_service: fastrpc_apps_user.c:4278: fastrpc_apps_user_init done  with default domain:3 and &fastrpc_trace:0x5585b86a80
trustedvm-v2 TVMRoot[704]: ADSPRPC: 2 3 I DSP_service: fastrpc_apps_user.c:1570: remote_handle_open_domain: attaching to securePD
trustedvm-v2 TVMRoot[704]: ADSPRPC: 2 3 I DSP_service: fastrpc_apps_user.c:1680: remote_handle64_open: opened handle 0x7f940036b0 (remote 0x0) for '";./\createstaticpd:securepd&_dom=cdsp on domain 3 (spa
trustedvm-v2 TVMRoot[704]: ADSPRPC: 2 3 I DSP_service: fastrpc_apps_user.c:3405: close_device_node: closed dev 8 on domain 3
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.woo@lge.com

Section 5

Hexagon Tensor Processor Use Cases

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Hexagon Tensor Processor Customization Areas

Area	Use case	Customization
Camera	▪ AI-Camera ▪ Face detection	–
FastCV (memory to memory)	FastCV HVX-optimized APIs	APIs are most widely used, most computationally intense, and have better power and performance Detailed list is available in Hexagon SDK documentation for FastCV tool `{HEXAGON_SDK_ROOT}\<version>\addons\compute\docs\fastcv\fastcv.html`
Machine learning	▪ Snapdragon® Neural Processing Engine (SNPE) SDK ▪ Qualcomm® Neural Network (QNN) SDK	SNPE/QNN SDK is available through the QPM

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Hexagon Tensor Processor Use Cases – Customization Workflow

Hexagon Tensor Processor Customization Stages

Evaluate	Define/Design	Integrate	Testing/Optimization	Manufacture
		Integration	**Power and Performance Optimization**	**Stability and Signing**
Customer downloads latest Hexagon SDK and LLVM toolchain (HVX examples are provided) for use case evaluation	Customer identifies feature API to implement on HVX; starts developing code using SDK while referring to SDK example code	Customer integrates and modifies HAL code to invoke HVX-optimized functionality	Customer optimizes power and performance	Customer resolves stability issues and signs module
Latest Hexagon SDK, toolchain, and examples are available on QPM	Customer develops standalone test app that exercises custom HVX-optimized functionality; verifies application on a target device. Before verification, target device must be signed using **testsig**. Customer includes performance profiling in this process	Customer performs additional end-to-end testing of feature using real-life use cases and profiles for power and performance requirements	Customer can use **sysmon** tool to profile performance	Customer can use **adspcrashman** to resolve stability issues; see the stability and debugging documentation. Static hashing to be used for mass production signing of module

Qualcomm Confidential - May Contain Trade Secrets
2024-05-21 02:30:08 GMT
hyungchul.kim@lge.com

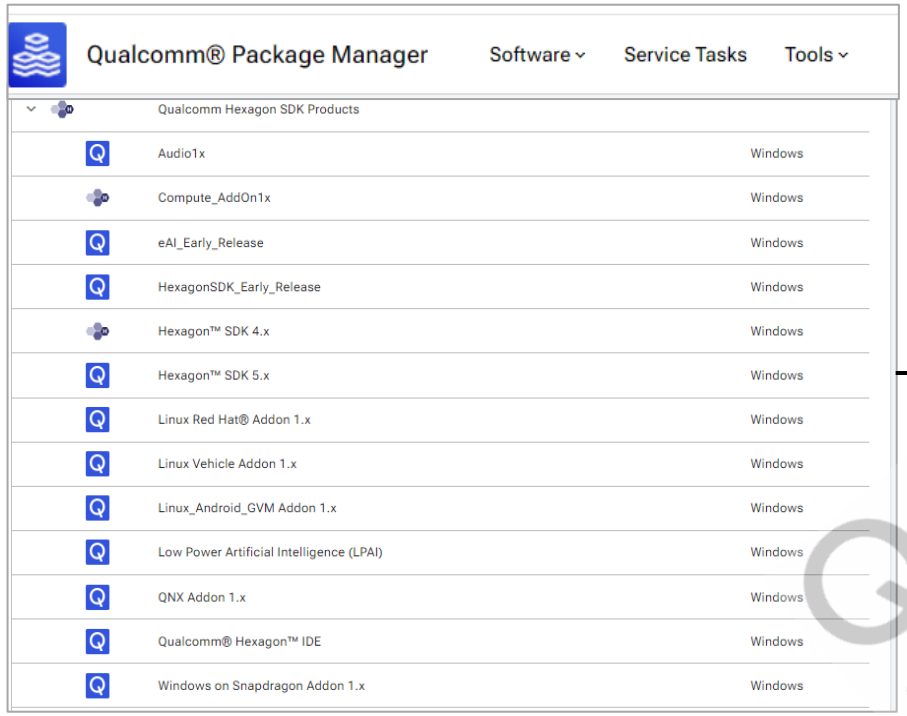
Section 6

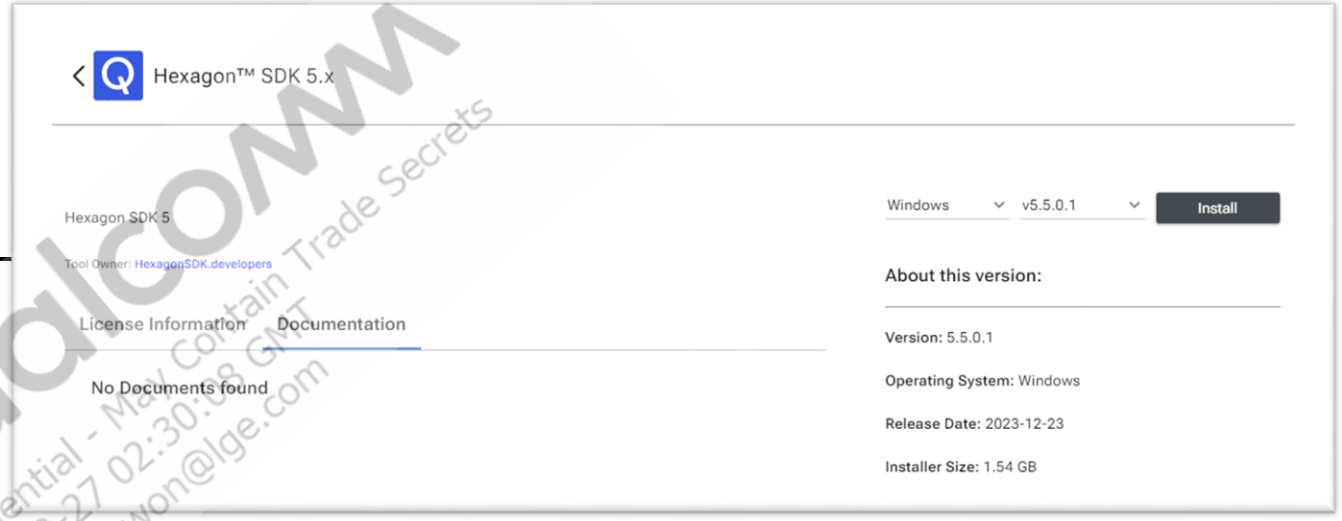
Hexagon Tensor Processor Development Environment

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – Development Tools – Hexagon SDK – Installation

Use the QPM application to install the Hexagon SDK product suite





Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – Development Tools – Hexagon SDK

- Hexagon SDK 6.0 latest release + latest release Add-on
- Linux and Windows development platforms
- Installation with QPM v3 support
- Hexagon IDE extensions for Visual Studio Code for building, debugging, and profiling examples on simulator and target
- Command-line build environment using Makefiles and CMake
- IDL compiler
  - For remote APIs
- Documentation
  - In-depth technical articles for Hexagon Tensor Processor with links to PDF documents
- Examples
  - Designed for compute and neural network use cases that can be compiled for simulator or target
  - Implemented in C/C++, HVX intrinsics, and assembly
  - Android application to demonstrate high-level features
  - Synxexample to demonstrate use of Synx APIs
- Tools:
  - LLVM compiler toolchain
    - Provides examples that can be modified, built, profiled, and debugged on PC
  - Simulator
    - For profiling DSP code performance using DSP OS-to-QuRT simulation
    - QEMU and Hexagon simulator support
  - Debugger for target and simulator
    - Multiple DSP debugging support
  - Profiling
    - Target: Hexagon trace analyzer and sysMon
    - Simulator: Hexagon profiler
    - Itrace profiling: Monitor CPU and HTP events
  - Signing
    - Scripts for generating development test signatures and signing production shared objects
    - Shared objects are authenticated prior to loading on Hexagon Tensor Processor
- Library to help port floating-point arithmetic

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 02:30:08 GMT hyunghul.won@lge.com

Development Environment – Hexagon SDK – Getting Started

- Start with benchmark example, which demonstrates multiple computational kernels for HVX and best practices:

  ${HEXAGON_SDK_ROOT}\<version>\addons\compute\docs\examples\benchmark\README.html

- For descriptions of command line parameters and usage instructions, see:

  ${HEXAGON_SDK_ROOT}\<version>\addons\compute\docs\examples\benchmark\README.html

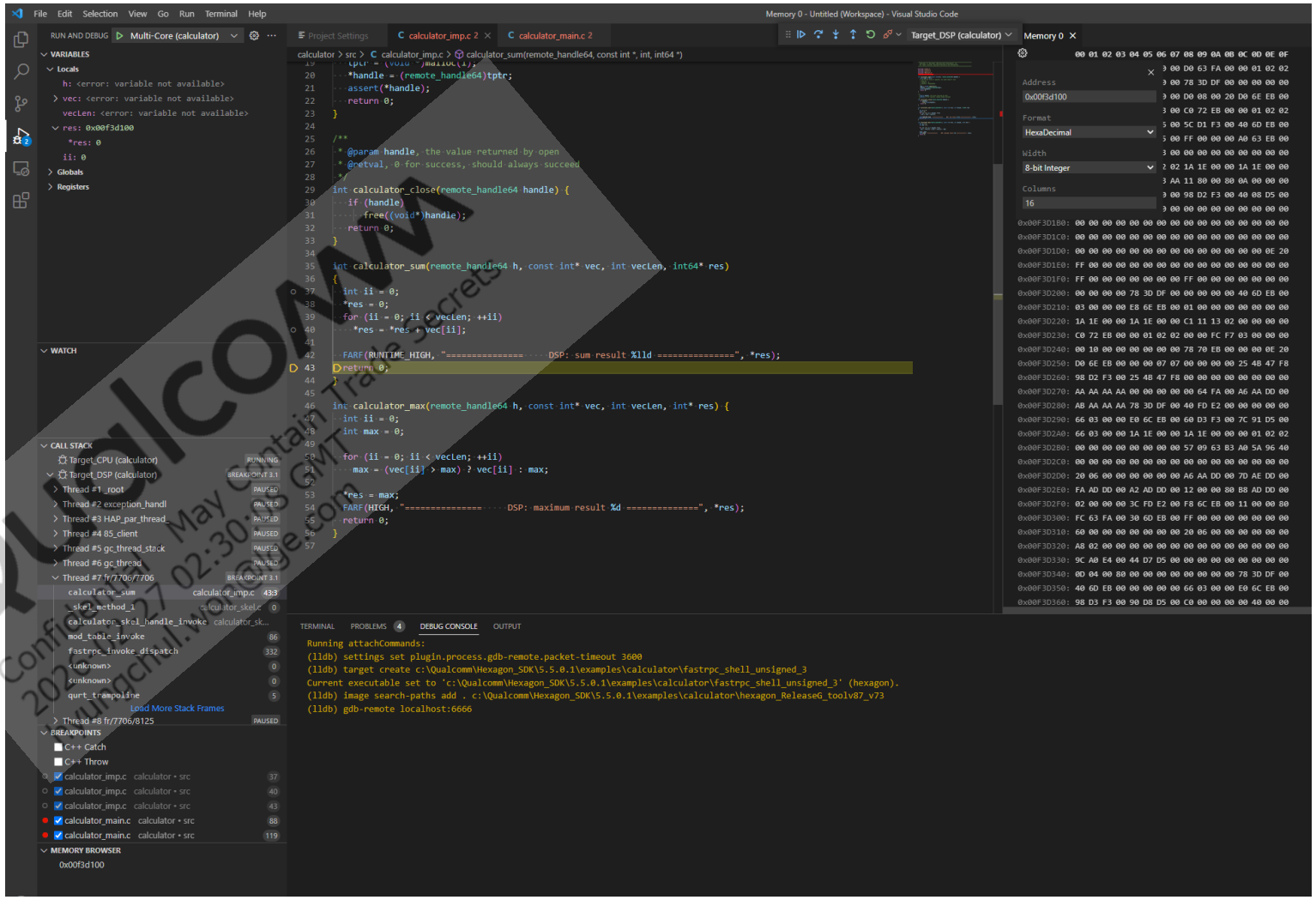
- To build the example and load the shared object to target, run:

  ${HEXAGON_SDK_ROOT}\<version>\addons\compute\examples\benchmark\benchmark_walkthrough.py

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

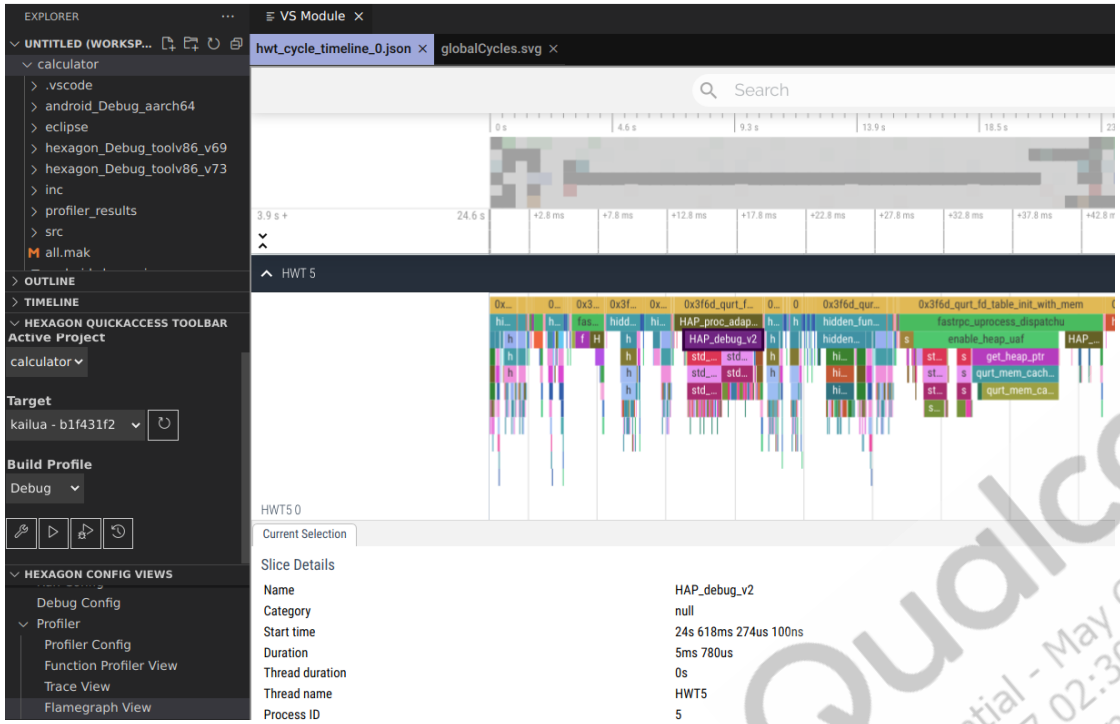
Development Environment – Hexagon SDK – Hexagon VSCode IDE

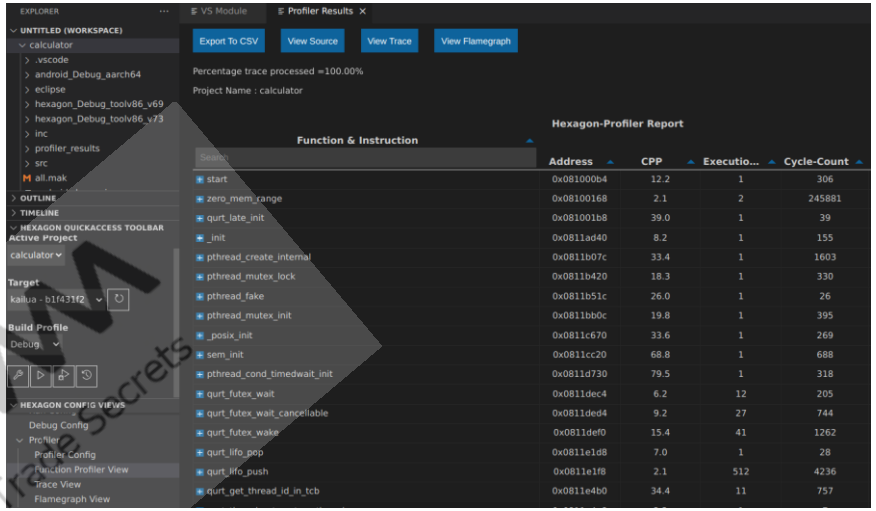
- Android/Hexagon build, debug, test, and profiling support
- Multi-core debug support
- Full C/C++ source and assembly-level debugging with Hexagon LLDB debugger for DSP and GDB debugger for CPU
- Global, local, and register view
- Memory browsing and editing with variable name or address
- Logging with logcat and mini-dm
- PD dump analysis in IDE
- Profiling on simulator and target
- Plug-ins

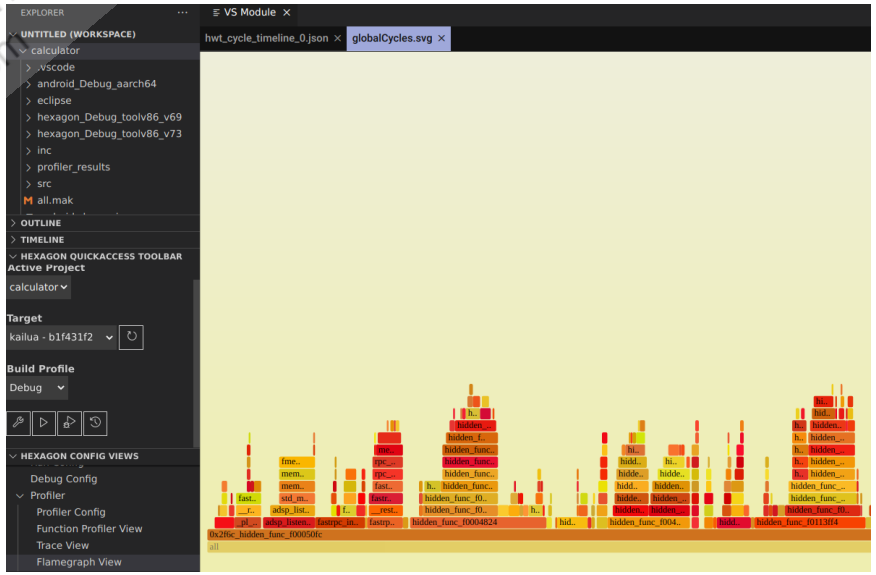


Qualcomm Confidential May contain Trade Secrets 2025-07-27 02:30:

Development Environment – Hexagon SDK – Hexagon VSCode IDE Profiling







Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Video Links for Hexagon VSCode IDE:

- Install on Windows
- Install on Linux
- Getting Started with Hexagon IDE
- Debugging with Hexagon IDE

Development Environment – Debugging – Debug Logs

- Debug logs (on target)
  - `FARF()` or `printf()` – Diagnostic messages on DSP can be sent using FARF or printf
  - Runtime FARF – Enable messages typically disabled in compilation (less overhead)
  - Logcat – Regular FARF messages on DSP can be directed to logcat

    When enabled, FARF messages on DSP show up in logcat stream with the tag *adsprpc*
  - The following diagnostic tools can be used to view and collect DSP messages:
    - QXDM Professional™ Tool (QXDM Professional) – For debug messages and available on CreatePoint
    - Mini-DM – Available as part of Hexagon SDK installation

      `${HEXAGON_SDK_ROOT}\<version>\tools\debug\mini-dm\WinNT`
    - For details, see `${HEXAGON_SDK_ROOT}\<version>\docs\tools\messaging.html`
- Examples
  - `cdsprpcd.farf`
    - `0x001000080004001f fastrpc_loader.c,fastrpc_kprocess.c`
  - `calculator.farf`
    - `0x001000080004001f fastrpc_loader.c, fastrpc_uprocess.c`

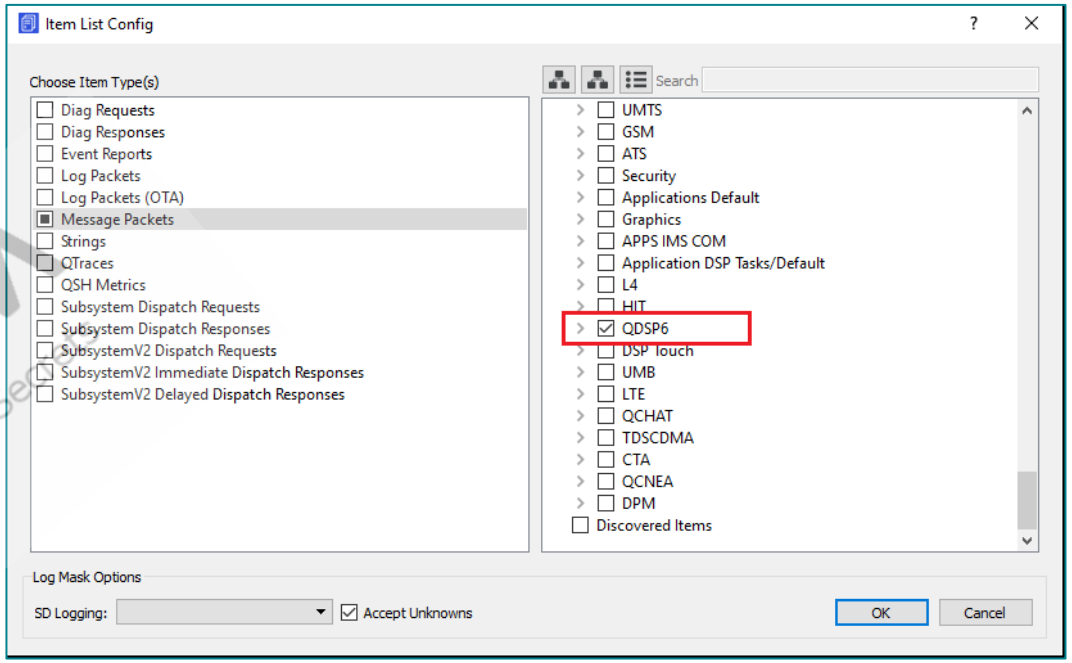
That is: `0x 00100008 0004001f`

User space | DSP space

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:00 GMT hyungchul.won@lge.com

Development Environment – Debugging – Debug Logs (cont.)

- QXDM Professional
  - **Messages View** shows DSP log message
  - In QXDM Professional, enable DSP log mask:
    1. Press **F3**.
    2. Right-click in the **Messages View** window.
    3. Select **Configure** > **Message Packets**.
    4. Select **QDSP6**.



- mini-dm
  - mini-dm shows the same log as QXDM Professional
  - Recommended if unfamiliar with QXDM Professional
  - In device manager or Qualcomm Product Support Tool (QPST), check comport number of USB diagnostic port
  - Close QXDM Professional and QPST, because Min-DM shares DIAG port

```
>mini-dm.exe --comport com120
<dm_serial_init> Opening port com120
<dm_serial_init> Port com120 opened!
mini-dm is waiting for a DMSS connection...
DMSS is connected. Running mini-dm...
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.wo@lge.com

Development Environment – Debugging – User Process Crash Log

- Debugging user process exceptions
  - When user process crashes on DSP, FastRPC exception handler collects required information from QuRT and flushes messages to logcat
- Logcat details
  - User process name
  - Thread name
  - Name of shared object and symbol offset (derived from PC during crash)
  - Kind of exception and details
  - Last known PC
  - Call trace

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – Debugging – User Process Crash Log (cont.)

```
00:33:50.450  402  sysmon_dcvs_clients.c:CDSP: DCVS client with client id: 7 is deregistered
00:33:50.453  792  05: CDSP:################################ Process on cDSP CRASHED!!!!!!! ##########################################
00:33:50.453  792  05: CDSP:--------------------- Crash Details are furnished below ------------------------------
00:33:50.453  792  05: CDSP:Process
00:33:50.453  792  05: CDSP:Crashed Shared Object
00:33:50.453  792  05: CDSP:fastrpc_shell_3 load address : A100000  and size : E863C
00:33:50.453  792  05: CDSP:Fault PC   :   0xA1187D4
00:33:50.453  792  05: CDSP:LR         :   0xA118A0C
00:33:50.453  792  05: CDSP:SP         :   0xC0896600
00:33:50.455  792  05: CDSP:Bad VA     :   0x0
00:33:50.455  792  05: CDSP:FP         :   0xC0896600
00:33:50.455  792  05: CDSP:SSR        :   0x1970427
00:33:50.455  792  05: CDSP:Error code :   0x2708
00:33:50.455  792  05: CDSP:Call trace :
00:33:50.455  398  01: CDSP:Error 0x8000040d: ftq_enqueue_on_group failed
00:33:50.455  792  05: CDSP:[<0A118A0C>] qurt_rmutex_unlock+0x4C:    (fastrpc_shell_3)
00:33:50.455  792  05: CDSP:[<0B93C214>] _Z25snpe_dsp_netrunner_deinitmP22DspUnloadNetworkInfo_tm+0x2E4:    (./libsnpe_dsp_v68_domains_v3_skel.so)
00:33:50.455  792  05: CDSP:[<0B93A2AC>] snpe_dsp_domains_v3_deinit+0x98:    (./libsnpe_dsp_v68_domains_v3_skel.so)
00:33:50.455  792  05: CDSP:[<0B938FDC>] snpe_dsp_domains_v3_skel_handle_invoke+0x66C:    (./libsnpe_dsp_v68_domains_v3_skel.so)
00:33:50.455  792  05: CDSP:[<0A182260>] remote_handle_close+0x31C8:    (fastrpc_shell_3)
00:33:50.455  792  05: CDSP:[<0A1727FC>] err_Fatal_internal0+0x7344:    (fastrpc_shell_3)
00:33:50.455  792  05: CDSP:[<0A178AAC>] err_Fatal_internal0+0xD5F4:    (fastrpc_shell_3)
00:33:50.455  792  05: CDSP:[<0A17CB34>] HAP_send_early_signal+0x1210:    (fastrpc_shell_3)
00:33:50.455  792  05: CDSP:---------------------------- End of Crash Report ----------------------------------------------
00:33:50.455  792  05: CDSP:Please refer to Hexagon SDK documentation
00:33:50.455  875  err_qdi.c:COMMON:flags = 0x0
```

1. Find the offset by calculating the difference between the Fault PC and load address of crashed shared object 0xA1187D4 - 0xA100000 = 0x187D4.
2. Using this offset and crashed shared object, run hexagon-addr2line for the line number in the source file where it crashed:
   hexagon-addr2line.exe -e fastrpc_shell_3 0x187D4
3. Run hexagon-llvm-objdump.exe to determine the disassembly of the crashing packet.
   hexagon-llvm-objdump.exe --disassemble -source fastrpc_shell_3 > out.txt
   Review the instruction for offset 0x187D4 in out.txt
4. cDSP: Error code: 0x2708
   0x08 is cause, and 0x27 is cause2
   *QURT_ERMUTEXUNLOCKNONHOLDER 39 /**< Rmutex unlock by a non-holder.*/ from Qurt_error.h*
   QURT_EXCEPT_EXIT 0x08 from Qurt_except.h

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 00:30:08 GMT kyongsho.yun@qualcomm.com

Development Environment – Debugging on Target Device Crash and Stability Issues

- PCAT
  - Crash dump collection
- Adspcrashman
  - RAM dump parsing
- TRACE32
  - Dump loading, scripts

Share the following files for cDSP crash debugging:

- DDR dump files
- Vmlinux
- cDSP symbol files (`target.*prod.elf`) under `cdsp_proc\build\ms`
- Linux kernel object files: `frpc-adsprpc.ko`, `cdsp-loader.ko`, `arm_smmu.ko`, `qcom_ipc_logging.ko`

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – Signing

- Unsigned PD
- Development signing
  - Algorithm development
  - Internal testing
- Production signing
  - Sub-CA
  - Static hashing

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – Unsigned PD

- Clients can request to offload signature-free dynamic shared objects to Hexagon Tensor Processor
- Signature-free dynamic shared object can run inside unsigned PD on Hexagon Tensor Processor
- Unsigned PD is sandboxed, low-rights process that allows signature-free modules to run on Hexagon Tensor Processor
  - In event of compromise, sandbox prevents access to full system functionality and data
  - Unsigned PDs are designed to support general compute applications and have limited access to underlying drivers
- Unsigned PD available services:
  - Thread creation and thread services
  - Timer creation and timer services
  - HVX contexts
  - Clock frequency controls
  - VTCM
  - Cache operations
  - Map high-level operating system (HLOS) memory allocated by corresponding HLOS application
  - QNN library

Qualcomm Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyunghul.won@lge.com

Development Environment – Unsigned PD (cont.)

- Unsigned PD limitations
  - Access to limited drivers (UBWC/DMA and camera streamer are not available to unsigned PDs)
- Request signature-free dynamic module offload; clients must make following request:

```
{
    struct remote_rpc_control_unsigned_module data;
    data.enable = 1;
    data.domain = CDSP_DOMAIN_ID;
    remote_session_control(DSPRPC_CONTROL_UNSIGNED_MODULE, (void*)&data, sizeof(data));
}
```

- Make request before calling any other FastRPC function
- Successful `remote_session_control` function allows clients to offload dynamic shared object to Hexagon Tensor Processor without signing

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.won@qc.com

Development Environment – Development Signing

- Algorithm development
  - Either use unsigned PD or generate a test signature file (`testsig.so`) and place in device file system before a standalone algorithm executable can be verified on target
  - `testsig.so` is specific to each device and must be generated at least once for a device before target testing
  - After a device is signed, updated or new shared objects do not require device to be signed again
  - Works only on non-production target hardware platform or devices
- Internal testing
  - Sign algorithm shared object
  - Useful for internal mass testing before going into production because `testsig.so` generation is not feasible for thousands of devices
  - Works only on non-production target hardware platform or devices

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:38 GMT
hyungchul.won@lge.com

Development Environment – Development Signing – Algorithm Development

1. Set up the environment:

```
${HEXAGON_SDK_ROOT}> setup_sdk_env.cmd
Setting up the Hexagon SDK environment locally
Done
```

2. Run the following command:

```
${HEXAGON_SDK_ROOT}\tools\scripts> python testsig.py
```

The following SO file is generated and placed in `\vendor\lib\rfsa\dsp\`:

```
testsig-<serial num>.so
```

For details on signing, see Hexagon SDK documentation:

`${HEXAGON_SDK_ROOT}\<version>\docs\tools\sign.html`

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.kim@lge.com

Development Environment – Development Signing – Internal Testing

- Elfsigner command to sign shared object:

```
python elfsigner.py -i INFILE -o [OUTDIR]
```

  - `INFILE` is the pathname of an ELF format shared object file to be signed
  - `OUTDIR` specifies the output folder pathname of signed output file
  - Ensure not to use this in combination with static hashing method

- Usage examples:

```
python elfsigner.py -i dynmod.so -o signed
```

  - Signs input file: `dynmod.so`
  - Writes signed output file to `./signed/dynmod.so`

```
python elfsigner.py -i input/dynmod.so -o signed
```

  - Signs input file: `./input/dynmod.so`
  - Writes signed output file to `./signed/dynmod.so`

- Only works with non-production devices

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyunchul.wonsige.com

Development Environment – Production Signing

- Static hashing method used for mass production signing of algorithm shared object, so it works only with specific DSP firmware image being shipped
- Algorithm shared object is hashed,
  - Hash key is stored in DSP firmware image upon recompilation
  - To authenticate, DSP firmware verifies hash key before loading algorithm-shared object at runtime
- DSP firmware must be recompiled when shared object implementation is changed

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – Production Signing (cont.)

1. Create nested directory `oem\build\dynamic_module_hash inside cdsp_proc.`

   `<cdsp build root>\cdsp_proc\oem\build\dynamic_modules_hash`

2. Copy the skel files for hashing to `dynamic_modules_hash.`
3. Export `OEM_ROOT=<cdsp build root>\cdsp_proc\oem.`
4. Compile the Hexagon Tensor Processor build:

   `<cdsp build root>\cdsp_proc\build\ms> python ./build_variant.py <target>.cdsp.prod`

   to clean the build: `python ./build_variant.py <target>.cdsp.prod --clean`

5. Flash cDSP images:

   `python cdsp_proc\config\scripts\flashcdsp.py`

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.yon@lge.com

Development Environment – Production Signing (cont.)

6. Verify that shared objects have been hashed correctly.
   a. Open :

   ```
   <cdsp build root>\cdsp_proc\core\kernel\devcfg\build\core_cdsp_root_libs\qdsp6\xxx.cdsp.prod\data\
   xxx_xml\devcfg.c
   ```

   b. Search for the shared object name to confirm that it appears with the hash code.

   c. Before compile:

   ```
   const StringDevice driver_list_xxx_xml[] = {
   {"/dev/ABTimeout",814297740u, 5740, NULL, 0, NULL },
   {"DirConnGpioInt",3283941420u, 5784, NULL, 0, NULL },
   {"/dev/NOCError",1518077100u, 5868, NULL, 0, NULL },
   {"/dev/NOCErrorOEM",462237293u, 5880, NULL, 0, NULL },
   {"PdcGpioInt",3177152790u, 5892, NULL, 0, NULL },
   {"/statichashes/libsysmon_skel.so",243161765u, 5904, NULL, 0, NULL },
   {"/statichashes/libsysmondomain_skel.so",362050205u, 6020, NULL, 0, NULL },
   {"/statichashes/libstabilitydomain_skel.so",582395817u, 6136, NULL, 0, NULL },
   ```

   d. After compile:

   ```
   const StringDevice driver_list_xxx_xml[] = {
   {"/dev/ABTimeout",814297740u, 5768, NULL, 0, NULL },
   {"/statichashes/libsysmon_skel.so",243161765u, 5780, NULL, 0, NULL },
   {"/statichashes/123_libsysmon_skel.so",3569570042u, 6128, NULL, 0, NULL },
   {"/statichashes/libsysmondomain_skel.so",362050205u, 6592, NULL, 0, NULL },
   {"/statichashes/libdsp_streamer_skel.so",117646021u, 6708, NULL, 0, NULL },
   {"/statichashes/123libsysmondomain_skel.so",1660329875u, 6824, NULL, 0, NULL },
   ```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 02:30:00 GMT hyungchul.hong@lge.com

Development Environment – Sub-CA

- CASS USB token (dongle) signing replaced by sub-CA
- USB token not renewed after expiration
- Customer should migrate to sub-CA and follow procedure listed in *Signing: Sub-CA* (80-PD867-113) for requesting and signing using sub-CA.
- No new agreement is required for customers who have already signed CASS agreement to get USB token
- New customers should follow up with Qualcomm Product Management team for CASS agreement details

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – Hexagon Library

- Hexagon Library (QHL) and QHL HVX
  - Provide HVX vectorized alternative to floating-point operations that are not feasible to convert to fixed-point operations
  - Significantly faster than scalar Hexagon floating-point unit
  - For details, see: `${HEXAGON_SDK_ROOT}\<version>\docs\examples\qhl\README.html`

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – Halide – An Image Processing Language

- Domain-specific programming language for writing high-performance image processing code on modern processors
  - Optimized image processing pipelines are difficult to write
- Halide depends on LLVM toolset to generate HVX binaries
  - Allows separation of algorithm from computational details
  - Enables rapid authoring and evaluation of optimized pipelines
- For more information, go to http://halide-lang.org

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – Programming in Halide

Separates *Algorithm* from *schedule*

Simple syntax to restructure computation

Programmers can quickly restructure computation to find optimal performance

Halide

0.9 ms/megapixel

```
Func box_filter_3x3(Func in) {
  Func blurx, blury;
  Var x, y, xi, yi;

  // The algorithm - no storage, order
  blurx(x, y) = (in(x-1, y) + in(x, y) + in(x+1, y))/3;
  blury(x, y) = (blurx(x, y-1) + blurx(x, y) + blurx(x, y+1))/3;

  // The schedule - defines order, locality; implies storage
  blury.tile(x, y, xi, yi, 256, 32)
       .vectorize(xi, 8).parallel(y);
  blurx.compute_at(blury, x).store_at(blury, x).vectorize(x, 8);

  return blury;
}
```

C++

0.9 ms/megapixel

```
void box_filter_3x3(const Image &in, Image &blury) {
  __m128i one_third = _mm_set1_epi16(21846);
  #pragma omp parallel for
  for (int yTile = 0; yTile < in.height(); yTile += 32) {
    __m128i a, b, c, sum, avg;
    __m128i blurx[(256/8)*(32+2)]; // allocate tile blurx array
    for (int xTile = 0; xTile < in.width(); xTile += 256) {
      __m128i *blurxPtr = blurx;
      for (int y = -1; y < 32+1; y++) {
        const uint16_t *inPtr = &(in[yTile+y][xTile]);
        for (int x = 0; x < 256; x += 8) {
          a = _mm_loadu_si128((__m128i*)(inPtr-1));
          b = _mm_loadu_si128((__m128i*)(inPtr+1));
          c = _mm_load_si128((__m128i*)(inPtr));
          sum = _mm_add_epi16(_mm_add_epi16(a, b), c);
          avg = _mm_mulhi_epi16(sum, one_third);
          _mm_store_si128(blurxPtr++, avg);
          inPtr += 8;
      }}
      blurxPtr = blurx;
      for (int y = 0; y < 32; y++) {
        __m128i *outPtr = (__m128i *)(&(blury[yTile+y][xTile]));
        for (int x = 0; x < 256; x += 8) {
          a = _mm_load_si128(blurxPtr+(2*256)/8);
          b = _mm_load_si128(blurxPtr+256/8);
          c = _mm_load_si128(blurxPtr++);
          sum = _mm_add_epi16(_mm_add_epi16(a, b), c);
          avg = _mm_mulhi_epi16(sum, one_third);
          _mm_store_si128(outPtr++, avg);
}}}}}
```

Complex loop nest with vector intrinsics

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

- For more information on Halide programming and Halide porting examples on HVX, see `${HEXAGON_SDK_ROOT}\<version>\tools\HEXAGON_Tools\<version>\Tools\Halide`

  or `${HEXAGON_SDK_ROOT}\<version>\Tools\Halide`

Development Environment – TVM Introduction

- Tensor Virtual Machine (TVM) is a compiler framework for machine learning
  - End-to-end compiler stack
  - Supports multiple hardware targets
- TVM supports Qualcomm HVX
  - Optimized for HVX hardware features
- TVM compiler builds Hexagon DSP binaries from the user-authored Python code
- TVM compiler code is portable to new Hexagon DSP hardware features
- TVM supports user-directed optimizations
  - Separates program into algorithm and schedule
- For more information, go to https://tvm.apache.org
- See *TVM for Hexagon User Guide* (80-PW212-1)

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:08 GMT hyungchul.won@lge.com

Development Environment – Programming with TVM

TVM

```python
def quantized_add_generic(size, a, b, a_offset, b_offset, a_mult, b_mult, output, target, ctx):
  # Construct the TVM computation.
  A_offset = tvm.var('A_offset', dtype='uint8')
  B_offset = tvm.var('B_offset', dtype='uint8')
  A_mult = tvm.var('A_mult', dtype='uint16')
  B_mult = tvm.var('B_mult', dtype='uint16')

  N = tvm.var("N")
  A = tvm.placeholder((N,), name='A', dtype='uint8')
  B = tvm.placeholder((N,), name='B', dtype='uint8')

  C = tvm.compute((A.shape),
        lambda i: ((A[i].astype('int32') - A_offset.astype('int32')) * A_mult.astype('int32')) +
                  ((B[i].astype('int32') - B_offset.astype('int32')) * B_mult.astype('int32')), name='C')

  # Create the schedule.
  s = tvm.create_schedule(C.op);
  px, x = s[C].split(s[C].op.axis[0], nparts=1)
  s[C].bind(px, tvm.thread_axis("pipeline"))

  # Construct the callable object "func" corresponding to the computation.
  func = tvm.build(s, [A, B, C, N, A_offset, B_offset, A_mult, B_mult], target, name='qadd_tvm')

  func(tvm.ndarray.array(a, ctx=ctx), tvm.ndarray.array(b, ctx=ctx), output,
       size, a_offset, b_offset, a_mult, b_mult)
```

Qualcomm Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.wan@lge.com

25 lines of Python vs. 601 lines of assembly

Enables HVX, NSP programming for all

No expertise in processor internals needed

Hand-authored assembly

```
340         {
341             r6=add(r4,#-16384)
342             r4=add(r4,#128)
343         }
344         {
345             v17:16.h=vsub(v8.ub,v4.ub)
346             v8.cur=vmem(r6+#0)
347         }
348         {
349             v19:18.w=vmpy(v16.h,v2.h)
350         }
351         {
352             v21:20.w=vmpy(v17.h,v2.h)
353             v23:22.w=vadd(v7:6.w,v19:18.w):sat
354         }
355         {
356             v27:26=vshuff(v23,v22,r1)
357             v25:24.w=vadd(v11:10.w,v21:20.w):sat
358         }
359         {
360             v29:28=vshuff(v25,v24,r1)
361         }
362         {
363             v31:30=vshuff(v28,v26,r1)
364         }
365         {
366             v9:8=vshuff(v29,v27,r1)
367             vmem(r2++#1)=v30
368         }
369         {
370             vmem(r2++#1)=v31
371         }
372         {
373             vmem(r2++#1)=v8
374         }
375         {
376             nop
377             vmem(r2++#1)=v9
378         }:endloop0
```

…562 more lines of assembly code not shown

Development Environment – UBWCDMA

- UBWCDMA IP provides DMA services to the Q6DSP module that enables it to access UBWC frames. The purpose of the UBWCDMA design is to perform translation between UBWC frames stored in DDR and linear frames stored in Q6DSP local memory (TCM implemented as locked L2 cache)
- Use case example: Read a Region of Interest (ROI) of a UBWC frame in DDR and store it in Q6DSP local memory in linear format. In this use case, Q6DSP issues a sequence of requests to read an ROI in a UBWC frame in DDR and stores the received ROI inside its local memory. Q6DSP performs the required data processing on the ROI with HVX, then moves on to the next ROI within the frame until the whole frame is processed

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Development Environment – SNPE/QNN

- SDK tools set up to abstract developers from hardware complexities, providing the advantage of fast, portable AI application development across multiple Snapdragon chipsets
- See *SM8750 AI/ML Software Overview* (80-64995-65) for details

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Section 7

References

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Documents and Standards

Title	Number
Hexagon V68 Programmer's Reference Manual	80-N2040-46
Hexagon V68 HVX Programmer's Reference Manual	80-N2040-47
Qualcomm FastCV Integration	80-NE989-7
Snapdragon Computer Vision Overview for MSM8996/MSM8998	80-NL315-9
Video Postprocessing on Linux Android Platforms	80-P9301-176
FastRPC Training	80-N7039-1
Hexagon aDSP and cDSP Stability Debugging Guide	80-N4597-2
Qualcomm HVX Software Resources	80-NV396-69
Hexagon Programming: Vector Extensions (HVX)	80-VB419-97
Hexagon HVX Streaming Customization for Camera	80-NF772-36
Halide for Hexagon Vector eXtensions (HVX) User Guide	80-PD002-1
Halide for Hexagon Vector eXtensions (HVX) Training	80-PD002-2
Hexagon gprof Profiler User Guide	80-N2040-29
Hexagon Secure cDSP Software Overview for SM8150	80-PK723-1
Qualcomm Snapdragon Neural Processing Engine (SNPE) Overview	80-NL315-18

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Documents and Standards (cont.)

Title	Number
Dynamic Loading in Secure PD	80-PR150-1
Hexagon Profiler User Guide	80-N2040-10
Qualcomm qprintf Library	80-VB419-109
Signing: Sub-CA	80-PD867-113
Computer Vision Processor (CVP) V1.x Programming Guide	80-PF777-91
Snapdragon Computer Vision Overview	80-NL315-9
cDSP HVX Programming Introduction	80-30789-7
SM8750/SM8750P AI/ML Software Overview	80-64995-65
TVM for Hexagon User Guide	80-PW212-1
Resources	
Hexagon SDK, Hexagon Toolchain	Hexagon_SDK\<*version*>\docs
CDSP Basic Debug Guide	KBA-191029015107

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Documents and Standards (cont.)

Hexagon Training Videos	
Course	**DCN**
Hexagon Programming: Processor Architecture	VD80-VB419-41A (Part 1) VD80-VB419-41B (Part 2) VD80-VB419-41C (Part 3) VD80-VB419-41D (Part 4) VD80-VB419-41E (Part 5) VD80-VB419-41F (Part 6)
Hexagon Programming: Software Development Tools	VD80-VB419-42A (Part 1) VD80-VB419-42B (Part 2) VD80-VB419-42C (Part 3) VD80-VB419-42D (Part 4) VD80-VB419-42E (Part 5) VD80-VB419-42F (Part 6)
Hexagon Programming: C Optimization	VD80-VB419-44A (Part 1) VD80-VB419-44B (Part 2) VD80-VB419-44C (Part 3) VD80-VB419-44D (Part 4) VD80-VB419-44E (Part 5) VD80-VB419-44F (Part 6) VD80-VB419-44G (Part 7)
Hexagon Programming: Assembly Programming	VD80-VB419-45A (Part 1) VD80-VB419-45B (Part 2) VD80-VB419-45C (Part 3) VD80-VB419-45D (Part 4) VD80-VB419-45E (Part 5)
Hexagon Programming: Memory System	VD80-VB419-60A (Part 1) VD80-VB419-60B (Part 2) VD80-VB419-60C (Part 3) VD80-VB419-60D (Part 4) VD80-VB419-60E (Part 5) VD80-VB419-60F (Part 6)

HVX Training Videos	
Course	**DCN**
Hexagon Core Review	VD80-P4366-1
HVX Architecture Review	VD80-P4366-2A (Part 1) VD80-P4366-2B (Part 2)
HVX Tools	VD80-P4366-3
Simple Example	VD80-P4366-4A (Part 1) VD80-P4366-4B (Part 2)
Unusual Features	VD80-P4366-5A (Part 1) VD80-P4366-5B (Part 2)
Exploring HVX Using 3x3 Convolution	VD80-P4366-6A (Part 1) VD80-P4366-6B (Part 2)
Microarchitecture	VD80-P4366-7A (Part 1) VD80-P4366-7B (Part 2)
Assembly Programming	VD80-P4366-8A (Part 1) VD80-P4366-8B (Part 2)
Assembly 3x3 Convolution Example	VD80-P4366-9
Advanced Algorithms	VD80-P4366-10A (Part 1) VD80-P4366-10B (Part 2) VD80-P4366-10C (Part 3)
Advanced Instructions – Data,	VD80-P4366-11A (Part 1) VD80-P4366-11B (Part 2) VD80-P4366-11C (Part 3)
Advanced Instructions – Arithmetic	VD80-P4366-12A (Part 1) VD80-P4366-12B (Part 2) VD80-P4366-12C (Part 3)
OS Considerations	VD80-P4366-13
HVX Scope	VD80-P4366-14

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Documents and Standards (cont.)

Halide Training Videos	
Course	**DCN**
Halide Programming	VD80-PG213-1A (Part 1) VD80-PG213-1B (Part 2) VD80-PG213-1C (Part 3) VD80-PG213-1D (Part 4)

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Acronyms or terms

Acronym or term	Definition
aDSP	Application digital signal processor
APC	Automatic power collapse
cDSP	Compute digital signal processor
CNN	Convolutional neural network
CV	Computer Vision
CVP	Computer Vision Processor
DCVS	Dynamic clock and voltage scaling
DMA	Direct memory access
DLC	Deep Learning Container
GPU	Graphics processing unit
HAL	Hardware abstraction layer
HAP	Hexagon Access Program
HLOS	High-level operating system
HVX	Hexagon Vector eXtensions
IDL	Interface description language
LLVM	LLVM is not an acronym; it is the full name of the project, which is a collection of modular and reusable compiler and toolchain technologies
PD	Protection domain

Acronym or term	Definition
QHL	Hexagon Library
QNN	Qualcomm Neural Network
QPST	Qualcomm Product Support Tool
ROI	Region of Interest
RPC	Remote procedure call
RTOS	Real time operating system
SCVE	Snapdragon Computer Vision Engine
SDK	Software Development Kit
SMT	Simultaneous Multi-Threading
SNPE	Snapdragon Neural Processing Engine
TCM	Tightly coupled memory
TVM	Tensor Virtual Machine
VLIW	Very long instruction word
VTCM	Vector TCM

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

Questions?

For additional information or to submit technical questions, go to: https://www.qualcomm.com/support

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:08 GMT
hyungchul.won@lge.com

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("**Qualcomm Technologies**"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "**Qualcomm Internal Use Only**", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com , the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

Qualcomm Confidential - May Contain Trade Secrets
2026-03-27 02:30:08 GMT
jyungchul.kim@lge.com